

SKRIPSI

**PENERAPAN REFAKTORISASI BERBASIS
KOMPONEN MODULAR UNTUK MENINGKATKAN
KINERJA DAN KUALITAS ARSITEKTUR PADA
APLIKASI WEB TVKU**



Disusun Oleh :

Nama : Rifqis Sakha Hilmi Aziz
NIM : A11.2022.14254
Program Studi : Teknik Informatika

**Fakultas Ilmu Komputer
Universitas Dian Nuswantoro
2025/2026**

PERSETUJUAN SKRIPSI

Nama : Rifqis Sakha Hilmi Aziz
NIM : A11.2022.14254
Program Studi : Teknik Informatika
Fakultas : Ilmu Komputer
Judul Tugas Akhir : Penerapan Refaktorisasi Berbasis Komponen
Modular Untuk Meningkatkan Kinerja Dan Kualitas
Arsitektur Pada Aplikasi Web TVKU

Tugas Akhir ini telah diperiksa dan disetujui,
Semarang, <tanggal *format DD MM YYYY*>

Menyetujui:
Pembimbing

Mengetahui:
Dekan Fakultas Ilmu Komputer

Gustina Alfa Trisnapradika, M.Kom
NPP. 0686.11.2015.821

Sri Winarno, Ph.D
NPP. 0686.11.1998.142

PENGESAHAN DEWAN PENGUJI

Nama : Rifqis Sakha Hilmi Aziz
NIM : A11.2022.14254
Program Studi : Teknik Informatika
Fakultas : Ilmu Komputer
Judul Tugas Akhir : Penerapan Refaktorisasi Berbasis Komponen
Modular Untuk Meningkatkan Kinerja Dan Kualitas
Arsitektur Pada Aplikasi Web TVKU

Tugas akhir ini telah diujikan dan dipertahankan dihadapan Dewan Penguji pada Sidang tugas akhir tanggal Menurut pandangan kami, tugas akhir ini memadai dari segi kualitas maupun kuantitas untuk tujuan penganugerahan gelar Sarjana Komputer (S.Kom)

Semarang, <tanggal *format DD MM YYYY*>
Dewan Penguji:

Adhitya Nugraha, S.Kom, M.CS
Anggota Penguji 1

Ardytha Luthfiarta, M.Kom
Anggota Penguji 2

Sri Winarno, Ph.D
Ketua Penguji

PERNYATAAN KEASLIAN SKRIPSI

Sebagai mahasiswa Universitas Dian Nuswantoro, yang bertanda tangan dibawah ini, saya:

Nama : Rifqis Sakha Hilmi Aziz

NIM : A11.2022.14254

Menyatakan bahwa karya ilmiah saya yang berjudul:

Penerapan Refaktorisasi Berbasis Komponen Modular Untuk Meningkatkan Kinerja Dan Kualitas Arsitektur Pada Aplikasi Web TVKU.

Merupakan karya asli saya (kecuali cuplikan dan ringkasan yang masing – masing telah saya jelaskan sumbernya). Apabila di kemudian hari, karya saya disinyalir bukan merupakan karya asli saya, yang disertai dengan bukti – bukti yang cukup, maka saya bersedia untuk dibatalkan gelar saya beserta hak dan kewajiban yang melekat pada gelar tersebut. Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Semarang

Pada tanggal : *<format DD MM YYYY>*

Yang menyatakan

(.....)

UCAPAN TERIMAKASIH

Dengan memanjatkan puji syukur kehadiran Allah SWT. Tuhan Yang Maha Pengasih dan Maha Penyayang yang telah melimpahkan segala rahmat, hidayah dan inayah-Nya kepada penulis sehingga laporan tugas akhir dengan judul “Penerapan Refaktorisasi Berbasis Komponen Modular Untuk Meningkatkan Kinerja Dan Kualitas Arsitektur Pada Aplikasi Web TVKU.” dapat penulis selesaikan sesuai dengan rencana karena dukungan dari berbagai pihak yang tidak ternilai besarnya. Oleh karena itu penulis menyampaikan terimakasih kepada:

1. Prof. Dr. Pulung Nurtantio Andono, S.T., M. Kom, selaku Rektor Universitas Dian Nuswantoro Semarang.
2. Sri Winarno Ph.D, selaku Dekan Fasilkom.
3. Edy Mulyanto, SSi, M.Kom, selaku Ka.Progdi Teknik Informatika.
4. Ajib Susanto, M.Kom, selaku pembimbing tugas akhir yang memberikan ide penelitian, memberikan informasi referensi yang penulis butuhkan dan bimbingan yang berkaitan dengan penelitian penulis.
5. Dosen-dosen pengampu di Fakultas Ilmu Komputer Teknik Informatika Universitas Dian Nuswantoro Semarang yang telah memberikan ilmu dan pengalamannya masing-masing, sehingga penulis dapat mengimplementasikan ilmu yang telah disampaikan.
6. Kedua orang tua beserta saudara-saudara saya yang selalu memberikan dukungan dan doa selama menyelesaikan skripsi ini.

Semoga Tuhan yang Maha Esa memberikan balasan yang lebih besar kepada beliau-beliau, dan pada akhirnya penulis berharap bahwa penulisan laporan tugas akhir ini dapat bermanfaat dan berguna sebagaimana fungsinya.

Semoga Tuhan Yang Maha Esa memberikan balasan yang lebih besar kepada beliau – beliau dan pada akhirnya penulis berharap bahwa penulisan laporan tugas akhir ini dapat bermanfaat dan berguna sebagaimana fungsinya

Semarang, Agustus 2024

Rifqis Sakha Hilmi Aziz

ABSTRAKSI

Penelitian ini berfokus pada implementasi refaktorisasi berbasis komponen modular untuk mengoptimalkan kinerja dan kualitas arsitektur pada aplikasi web TVKU. Permasalahan utama yang diatasi meliputi arsitektur legacy yang masih mengandalkan jQuery dan template HTML lama, manajemen state yang tidak terpusat mengakibatkan prop drilling tinggi, serta performa buruk dengan Largest Contentful Paint (LCP) 65% lebih lambat dari standar. Solusi yang diimplementasikan mencakup migrasi ke Next.js 14 dengan penerapan React Query untuk data fetching, Zustand untuk state management terpusat, dan Tailwind CSS untuk utility-first styling. Metodologi Extreme Programming diterapkan dalam proses pengembangan, sementara evaluasi hasil menggunakan pengujian komprehensif meliputi unit testing, integration testing, dan performance testing dengan Lighthouse metrics. Hasil yang diharapkan adalah peningkatan signifikan pada metrik Core Web Vitals, pengurangan bundle size, peningkatan maintainability kode, serta eliminasi technical debt warisan jQuery.

Kata kunci: refaktorisasi, komponen modular, arsitektur legacy, React Query, performa web, maintainability.

DAFTAR ISI

PERSETUJUAN SKRIPSI	ii
PENGESAHAN DEWAN PENGUJI	iii
PERNYATAAN KEASLIAN SKRIPSI.....	iv
UCAPAN TERIMA KASIH.....	v
ABSTRAKSI.....	vi
DAFTAR ISI.....	vii
DAFTAR GAMBAR	x
DAFTAR TABEL.....	xi
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Batasan Masalah.....	4
1.4 Tujuan Penelitian	4
1.5 Manfaat Penelitian	5
1.5.1 Bagi Pengembang / Tim Teknis:.....	5
1.5.2 Bagi Mitra TVKU:	5
1.5.3 Manfaat Akademis:	6
BAB II LANDASAN TEORI	7
2.1 Tinjauan Studi	7
2.1.1 Penelitian Terkait Refaktorisasi Sistem	7
2.1.2 Penelitian Terkait Manajemen Data Modern	8
2.1.3 Penelitian Terkait Efisiensi dan Kinerja Aplikasi	8
2.2 Tinjauan Pustaka	11
2.2.1 Arsitektur Sistem Berbasis Komponen	11
2.2.2 Refaktorisasi Perangkat Lunak pada Frontend Modern.....	15
2.2.3 Pola Aliran dan Manajemen Data Frontend.....	20
2.2.4 Desain UI Modern Berbasis Utility-First.....	23
2.2.5 Evaluasi dan Optimalisasi Kinerja Sistem Web.....	26
2.2.6 Aksesibilitas dan Pengalaman Pengguna (UX).....	30
2.2.7 UML dalam Analisis dan Perancangan Sistem	33
2.3 Kerangka Pemikiran.....	36
BAB III METODOLOGI PENELITIAN.....	37
3.1 Metode Pengumpulan Data	37

3.1.1 Observasi Langsung	37
3.1.2 Wawancara Terbatas	37
3.1.3 Studi Pustaka	38
3.1.4 Hasil Pengumpulan Data	38
3.2 Metode yang diusulkan	39
3.2.1 Deskripsi Umum Sistem	39
3.2.2 Tujuan Sistem yang Diusulkan	40
3.2.3 Arsitektur dan Teknologi yang Digunakan	41
3.3 Metode yang Diusulkan	42
3.3.1 Tahap Planning	43
3.3.2 Tahap Design	43
3.3.3 Tahap Coding	44
3.3.4 Tahap Testing	44
3.3.5 Tahap Release	45
3.4 Metode Pengujian	45
3.4.1 Pengujian Unit (Unit Testing)	45
3.4.2 Pengujian Integrasi (Integration Testing)	46
3.4.3 Pengujian Fungsional (Functional / Black-Box Testing)	46
3.4.4 Pengujian Performa (Performance Testing)	46
3.4.5 User Acceptance Testing (UAT)	47
BAB IV HASIL PENELITIAN DAN PEMBAHASAN	48
4.1 Analisis Sistem dan Kebutuhan	48
4.1.1 Gambaran Umum Objek Penelitian	48
4.1.2 Permasalahan Teknis Sistem Lama	48
4.2 Perancangan Sistem Modern	51
4.2.1 Perancangan Arsitektur Next.js 14	51
4.2.2 Pemodelan UML (Sistem Baru)	51
4.2.3 Perancangan Antarmuka Berbasis Atomic Design	59
4.2.4 Perancangan Manajemen State Terpisah	60
4.3 Tahapan Refaktorisasi Sistem	61
4.3.1 Restrukturisasi Proyek	61
4.3.2 Modularisasi Komponen (Before vs After):	62
4.3.3 Optimasi Performa Aplikasi	65
4.3.4 Implementasi Fitur Utama (Dokumentasi Antarmuka)	66

4.4 Pengujian dan Evaluasi	73
4.4.1 Pengujian Fungsional (Black Box Testing)	73
4.4.2 Pengujian Performa (Lighthouse & Core Web Vitals)	74
4.4.3 Pengujian Maintainability	74
4.4.4 User Acceptance Testing (UAT).....	75
BAB V KESIMPULAN DAN PENELITIAN SELANJUTNYA	77
5.1 Kesimpulan	77
5.2 Penelitian Selanjutnya.....	78
DAFTAR PUSTAKA	79

DAFTAR GAMBAR

Gambar 1. Struktur Umum Arsitektur Berbasis Komponen Aplikasi TVKU	13
Gambar 2. Hierarki Modul Aplikasi TVKU	14
Gambar 3. Alur Lifecycle Rendering	15
Gambar 4. Alur Proses Refaktorisasi Clinet Site TVKU	18
Gambar 5. Struktur Komponen Sebelum & Setelah Refaktorisasi	19
Gambar 6. Code Smell – Technical Debt – Refactoring.....	19
Gambar 7. Pola Aliran Data dalam Aplikasi Berbasis Komponen	22
Gambar 8. Arsitektur Konseptual Manajemen Data Clinet Site TVKU	22
Gambar 9. Atomic design hierarchy diagram	24
Gambar 10. Tailwind Css Pipeline Diagram.....	25
Gambar 11. Proses Evaluasi Kinerja Web Modern	29
Gambar 12. Optimalisasi Performa Frontend	30
Gambar 13. <i>Hubungan antara Aksesibilitas dan UX dalam Sistem Frontend TVKU.....</i>	32
Gambar 14. Hubungan Antar Digram UML	35
Gambar 15. Kerangka Pemikiran.....	36
Gambar 16. Struktur Arsitektur Umum Sistem Frontend TVKU yang Diusulkan	40
Gambar 17. Alur Interaksi Antar Modul dalam Sistem Frontend TVKU.....	40
Gambar 18. <i>Use Case Diagram Sistem TVKU.....</i>	52
Gambar 19. Activity Diagram Home	53
Gambar 20. Activity Diagram News.....	54
Gambar 21. Activity Diagram Seputar Udinus	55
Gambar 22. Activity Diagram Program	56
Gambar 23. Activity Diagram Live Streaming	57
Gambar 24. Activity Diagram Jadwal Siaran	58
Gambar 25. Use Case Diagram Sistem TVKU	59
Gambar 26. <i>Hierarki Atomic Design pada UI TVKU</i>	60
Gambar 27. Hierarki Folder Sesudah Refaktorisasi.....	61
Gambar 28. Halaman Beranda dan Navigasi sebelum Refactorisasi	66
Gambar 30. Halaman Beranda dan Navigasi setelah Refactorisasi	67
Gambar 30. Halaman Berita sebelum Refactorisasi	67
Gambar 31. Halaman Berita setelah Refactorisasi.....	68
Gambar 32. Halaman Program sebelum Refactorisasi	69
Gambar 33. Halaman Program setelah Refactorisasi.....	69
Gambar 34. Halaman Seputar UDINUS sebelum Refactorisasi.....	70
Gambar 35. Halaman Seputar UDINUS setelah Refactorisasi	71
Gambar 36. Halaman Live Streミング sebelum Refactorisasi	71
Gambar 37. Halaman Live Streミング setelah Refactorisasi	72
Gambar 38. Halaman Modul Jadwal Siaran sebelum Refactorisasi	72
Gambar 39. Halaman Modul Jadwal Siaran setelah Refactorisasi	73

DAFTAR TABEL

Tabel 1. Perbandingan Penelitian Terdahulu Terkait Refaktorisasi dan Kinerja Sistem.....	9
Tabel 2. Ringkasan Temuan Empiris Terkait Peningkatan Kinerja Sistem.....	11
Tabel 3. Jenis dan Tujuan Refaktorisasi dalam Rekayasa Perangkat Lunak	17
Tabel 4. Perbandingan Teknologi Pengelolaan Data Modern	21
Tabel 5. Perbandingan Framework CSS Modern	24
Tabel 6. Nilai Ideal Core Web Vitals	26
Tabel 7. Teknik Optimasi Kinerja Frontend Modern.....	27
Tabel 8. Perbandingan Elemen Aksesibilitas dan Pengalaman Pengguna (UX) ..	31
Tabel 9. Jenis dan Konteks Penggunaan Diagram UML pada Sistem TVKU.....	34
Tabel 10. Teknologi yang Digunakan dalam Sistem yang Diusulkan	41
Tabel 11. Permasalahan teknis legacy	49
Tabel 12. <i>Kebutuhan Fungsional dan non fungsional</i>	50
Tabel 13. Perbandingan Pola Kode Legacy vs Modern	62
Tabel 14. Hasil Black Box Testing	73
Tabel 15. Metrik Performa (Lighthouse)	74

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi mendorong institusi media, termasuk televisi komunitas, untuk mengoptimalkan layanan berbasis web sebagai sarana distribusi informasi dan hiburan. Aplikasi berbasis web tidak hanya dituntut mampu menampilkan konten multimedia secara andal, tetapi juga harus mendukung ketersediaan informasi secara cepat, konsisten, serta mudah diakses pada berbagai perangkat. Hal ini menempatkan lapisan antarmuka pengguna sebagai elemen strategis yang berperan penting dalam menentukan kualitas pengalaman pengguna dan daya saing aplikasi digital (Duarte et al., 2023).

TVKU, sebagai stasiun televisi komunitas di bawah Universitas Dian Nuswantoro, telah mengembangkan aplikasi berbasis web untuk menyajikan layanan livestreaming, portal berita, dan informasi program. Namun, sistem tampilan yang digunakan masih memadukan kerangka kerja modern berbasis React dan Next.js dengan elemen warisan berupa template HTML, CSS, dan pustaka jQuery. Integrasi semacam ini menimbulkan persoalan mendasar, karena paradigma manipulasi Document Object Model (DOM) secara langsung oleh jQuery bertentangan dengan pendekatan deklaratif berbasis state yang digunakan dalam React. Ketidaksesuaian paradigma ini kerap menjadi sumber inkonsistensi tampilan dan kesalahan logika pada interaksi pengguna (Serrano et al., 2022).

Selain itu, pengelolaan data pada aplikasi TVKU belum mengikuti praktik terbaik mutakhir. Data ditangani secara imperatif di dalam komponen dengan memanfaatkan hook bawaan React seperti `useState` dan `useEffect`, tanpa mekanisme manajemen state yang terpusat maupun strategi caching. Kondisi ini berdampak pada keterlacakan alur data yang lemah, tingginya risiko redundansi kode, dan kerentanan terhadap prop drilling—fenomena ketika data

harus diteruskan secara eksplisit dari satu komponen ke komponen lain dalam hierarki yang panjang (Zhou & Kim, 2021).

Dari sisi pemeliharaan, penggunaan berkas CSS monolitik yang bersumber dari tema lama menimbulkan kompleksitas tersendiri. Konflik antar aturan gaya, sulitnya kustomisasi, dan tingginya keterkaitan antar modul menjadikan kode sulit untuk dikembangkan maupun diperluas. Penelitian sebelumnya menunjukkan bahwa sistem tampilan yang tidak modular cenderung meningkatkan technical debt dan menurunkan efisiensi pengembangan perangkat lunak (Nugroho et al., 2022).

Kondisi ini juga berimplikasi pada performa. Kebergantungan pada pustaka jQuery dan berkas CSS/JavaScript berukuran besar dari template lama menyebabkan ukuran berkas aplikasi (bundle size) meningkat signifikan. Studi terbaru menemukan bahwa penggunaan jQuery dalam aplikasi berbasis web dapat memperburuk metrik performa seperti Largest Contentful Paint (LCP) hingga 65% dibandingkan dengan aplikasi yang sepenuhnya memanfaatkan arsitektur modern tanpa jQuery (WordPress Performance Team, 2021). Bahkan pada kasus GOV.UK, penghapusan jQuery berhasil mempercepat waktu interaktivitas halaman hingga 17% (Government Digital Service, 2022). Lebih jauh, Menurut Google Chrome UX Report Team (2023), keterlambatan pemuatan satu detik dapat menurunkan kepuasan pengguna sebesar 15–20% dan tingkat konversi hingga 7%, sebagaimana ditunjukkan dalam pembaruan Core Web Vitals yang menyoroti pentingnya Largest Contentful Paint (LCP) dan First Input Delay (FID) terhadap pengalaman pengguna. Fakta ini mempertegas bahwa ketergantungan pada pustaka inheritance tidak hanya menghambat maintainability, tetapi juga secara nyata menurunkan performa aplikasi.

Untuk menjawab tantangan tersebut, diperlukan upaya rekayasa ulang pada lapisan antarmuka pengguna aplikasi TVKU dengan mengadopsi arsitektur modern. Salah satu pendekatan yang kini banyak digunakan adalah pemisahan manajemen state antara server state dan client state melalui pustaka khusus seperti React Query dan Zustand. React Query menyediakan mekanisme

pengambilan dan sinkronisasi data secara deklaratif dengan dukungan caching serta penanganan loading dan error state, sedangkan Zustand memberikan pengelolaan state global yang ringan dan efisien untuk kebutuhan antarmuka pengguna (Gupta & Sharma, 2024).

Selain itu, pendekatan utility-first styling yang ditawarkan oleh Tailwind CSS semakin populer karena terbukti mampu menekan kompleksitas kode CSS, meningkatkan konsistensi desain, dan mempercepat proses pengembangan antarmuka (Rahman et al., 2023). Dengan kombinasi teknologi modern tersebut, diharapkan tercapai peningkatan signifikan dalam maintainability dan performa aplikasi.

Berdasarkan latar belakang tersebut, penelitian ini berfokus pada upaya refaktorisasi lapisan antarmuka pengguna aplikasi TVKU dengan tujuan meningkatkan kualitas arsitektur perangkat lunak, mengurangi ketergantungan pada teknologi usang, serta mengoptimalkan performa dan pengalaman pengguna.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan, permasalahan dalam penelitian ini dapat dirumuskan sebagai berikut:

1. Bagaimana merancang ulang lapisan antarmuka pengguna aplikasi TVKU agar tidak lagi bergantung pada teknologi warisan seperti jQuery, serta sepenuhnya memanfaatkan paradigma arsitektur berbasis komponen yang modular dan konsisten?
2. Bagaimana penerapan strategi pengelolaan server state melalui React Query dan client state melalui Zustand dapat meningkatkan maintainability serta keterlacakan alur data aplikasi?
3. Bagaimana membandingkan metrik teknis seperti rendering dan ukuran berkas aplikasi sebelum dan sesudah dilakukan refaktorisasi, sehingga dapat diketahui sejauh mana arsitektur modern memberikan peningkatan performa?

1.3 Batasan Masalah

Untuk menjaga agar penelitian ini fokus dan dapat diselesaikan dengan baik, penelitian ini dibatasi pada aspek-aspek berikut:

1. Penelitian hanya berfokus pada Client side aplikasi TVKU. Tidak mencakup perubahan di Server side, API, ataupun infrastruktur server.
2. Refactoring meliputi: penghapusan penggunaan jQuery; perombakan struktur komponen front end; pemisahan server state dan client state; penerapan styling menggunakan Tailwind CSS; dan penggunaan React Query + Zustand untuk data fetching dan state management.
3. Penelitian ini tidak mencakup pengembangan maupun evaluasi fitur dengan hak akses administratif (role admin). Pembatasan ini bertujuan untuk menjaga fokus penelitian pada optimalisasi performa dan kualitas arsitektur aplikasi dari sisi pengguna akhir (client side).
4. Pengukuran performa dibandingkan antara kondisi sebelum refactoring dan setelah refactoring menggunakan alat, alat seperti Lighthouse, Next.js Bundle Analyzer, dan/atau alat profiling lainnya.
5. Penelitian ini berfokus pada metrik teknis seperti Largest Contentful Paint (LCP), Time to Interactive (TTI), First Input Delay (FID), dan Cumulative Layout Shift (CLS) untuk mengevaluasi kinerja sistem. Selain itu, aspek maintainability juga menjadi perhatian utama, yang diukur melalui praktik Code Clean-up, Modularization, Dependency Removal, dan State Refactoring. Pendekatan ini dipilih sebagai alternatif pengukuran kepuasan pengguna (user survey) secara luas, meskipun survei berskala kecil dapat dipertimbangkan sebagai pelengkap jika memungkinkan.

1.4 Tujuan Penelitian

Dari penelitian ini ada beberapa hal yang menjadi tujuan peneliti, antara lain adalah:

1. Mengimplementasikan arsitektur frontend modern untuk aplikasi TVKU yang mengeliminasi ketergantungan terhadap jQuery dan memanfaatkan komponen modular, React Query, Zustand, dan Tailwind CSS.

2. Menganalisis dan mendokumentasikan peningkatan maintainability setelah refactoring, meliputi aspek struktur kode, modularitas, kemudahan debugging, dan pengelolaan state.
3. Mengukur perbandingan performa teknis (bundle size, waktu muat awal, TTI, LCP, dan metrik performa lainnya) sebelum dan sesudah refactoring.
4. Menilai dampak penggunaan styling utility-first (Tailwind CSS) terhadap ukuran CSS, konflik styling, dan efisiensi pengembangan desain UI.
5. Memberikan rekomendasi praktis bagi pengembang aplikasi media berbasis web mengenai arsitektur frontend modern agar maintainability dan performa dapat tercapai secara optimal.

1.5 Manfaat Penelitian

Berikut manfaat yang diperoleh dalam melakukan penelitian ini antara lain:

1.5.1 Bagi Pengembang / Tim Teknis:

- a. Memperoleh panduan konkret tentang bagaimana merombak frontend legacy menjadi lebih modular, mudah dipelihara, dan optimal.
- b. Mengurangi teknis hutang (technical debt), bug yang muncul akibat konflik DOM/styling, serta mempermudah fitur tambahan dan pemeliharaan.
- c. Peningkatan kecepatan pengembangan UI dan perbaikan desain karena penggunaan styling yang konsisten (Tailwind) dan state management yang terorganisir.

1.5.2 Bagi Mitra TVKU:

- a. Performa aplikasi yang lebih baik: load page lebih cepat, interaktivitas lebih responsif, kemudahan akses terutama pada perangkat/peramban lambat atau koneksi rendah.
- b. Penghematan sumber daya: bundle yang lebih kecil, mengurangi penggunaan bandwidth, serta memori/CPU di sisi klien.
- c. Produk yang lebih profesional dan ramah pengguna, yang dapat meningkatkan reputasi stasiun televisi komunitas dan kepuasan penonton.

1.5.3 Manfaat Akademis:

- a. Kontribusi dalam literatur pengembangan web dan rekayasa perangkat lunak mengenai praktik refactoring frontend, penggunaan teknologi modern seperti React Query, Zustand, dan utility-first styling.
- b. Data empiris terbaru yang membandingkan kondisi legacy vs modern, yang bisa menjadi acuan penelitian selanjutnya.
- c. Perluasan kajian performa frontend modern termasuk pengaruh bundle size, state management, dan styling terhadap pengalaman pengguna.

BAB II

LANDASAN TEORI

2.1 Tinjauan Studi

2.1.1 Penelitian Terkait Refaktorisasi Sistem

Zhou dan Kim (2021) melakukan analisis terhadap pengelolaan state pada kerangka kerja berbasis komponen dan menemukan bahwa pemisahan state secara terpusat mampu menurunkan kompleksitas serta mengurangi risiko duplikasi logika data. Serrano et al. (2022) membandingkan paradigma manipulasi Document Object Model (DOM) langsung melalui jQuery dengan pendekatan deklaratif modern. Hasilnya menunjukkan peningkatan konsistensi tampilan dan penurunan beban proses render sebesar 23%.

Nugroho et al. (2022) menyoroti tantangan maintainability dalam kode JavaScript berskala besar. Mereka menemukan bahwa penerapan struktur modular menurunkan tingkat technical debt hingga 41%. Selanjutnya, Kumar et al. (2023) meneliti refaktorisasi antarmuka lama menuju arsitektur modern dan melaporkan peningkatan performa Time to Interactive (TTI) rata-rata 30%.

Refaktorisasi sistem warisan juga dibahas oleh Mendez et al. (2021) yang menegaskan pentingnya penghapusan dependensi lama untuk memperpanjang umur sistem dan memudahkan integrasi teknologi baru. Sementara itu, Prasetyo dan Hidayat (2022) menunjukkan bahwa restrukturisasi proyek berbasis JavaScript dapat mengurangi code smell sebesar 38% serta meningkatkan kecepatan pengembangan fitur baru hingga 26%.

Berdasarkan tinjauan terhadap penelitian-penelitian sebelumnya, masih terdapat peluang penelitian pada konteks refaktorisasi sistem berbasis web yang diterapkan pada lingkungan media lokal. Sebagian besar riset terdahulu berfokus pada sistem berskala besar atau aplikasi komersial, sedangkan penerapan refaktorisasi pada sistem informasi media berbasis

konten seperti TVKU belum banyak dieksplorasi. Oleh karena itu, penelitian ini berpotensi memberikan kontribusi empiris dalam menguji efektivitas refaktorisasi modular terhadap peningkatan kualitas dan kinerja sistem media lokal di Indonesia.

2.1.2 Penelitian Terkait Manajemen Data Modern

Gupta dan Sharma (2024) melakukan studi komparatif terhadap Redux dan Zustand sebagai pustaka manajemen state. Hasilnya, Zustand dinilai lebih efisien dalam konsumsi memori dan lebih mudah diintegrasikan dengan asynchronous data flow.

Tan et al. (2022) meneliti implementasi React Query untuk pengelolaan server state. Mekanisme caching dan automatic refetching yang disediakan terbukti menekan permintaan jaringan hingga 42 %, mempercepat proses render awal sebesar 18 %, serta menurunkan beban server.

Almeida et al. (2020) menegaskan bahwa strategi client-side state management yang baik mampu meningkatkan keterlacakan alur data dan mengurangi prop drilling dalam struktur komponen yang kompleks. Penelitian lain oleh Rafi et al. (2022) menunjukkan bahwa pemisahan tanggung jawab antara state logic dan view logic memperkuat modularitas sistem serta memudahkan pengujian unit.

Berdasarkan hasil kajian pustaka, masih terdapat ruang penelitian terkait implementasi manajemen data modern pada sistem berskala menengah. Penelitian sebelumnya cenderung berfokus pada lingkungan e-commerce atau aplikasi sosial, sementara konteks sistem media lokal belum banyak mendapat perhatian. Penelitian ini membuka peluang untuk mengevaluasi integrasi pustaka manajemen data modern dalam meningkatkan efisiensi pemrosesan dan stabilitas sistem berbasis komponen.

2.1.3 Penelitian Terkait Efisiensi dan Kinerja Aplikasi

Dalam konteks performa aplikasi, WordPress Performance Team (2021) menemukan bahwa penggunaan jQuery memperburuk metrik Largest Contentful Paint (LCP) hingga 65 %. Government Digital Service (2022) membuktikan bahwa penghapusan jQuery di GOV.UK meningkatkan

waktu interaktivitas halaman sebesar 17 %. Sementara itu, Google Chrome UX Report Team (2023) melaporkan bahwa peningkatan Largest Contentful Paint (LCP) di atas 2,5 detik dapat menurunkan kepuasan pengguna hingga 20 % dan berdampak langsung pada penurunan tingkat konversi sebesar 7 %.

Rahman et al. (2023) mengevaluasi pendekatan utility-first pada pengelolaan gaya. Penggunaan Tailwind CSS mampu menurunkan ukuran berkas CSS hingga 70 % dan mempercepat waktu pengembangan. Duarte et al. (2023) juga menegaskan bahwa pola arsitektur yang memisahkan logika bisnis, data, dan tampilan berpengaruh langsung terhadap efisiensi pemeliharaan dan kinerja sistem.

Nakamura et al. (2023) mengidentifikasi bahwa optimalisasi bundle size berkontribusi signifikan terhadap First Input Delay (FID), dimana pengurangan 10 % ukuran berkas menghasilkan peningkatan responsivitas hingga 8 %. Temuan ini memperkuat dasar pentingnya penghapusan pustaka usang serta penggunaan pustaka modern yang mendukung tree-shaking dan lazy-loading.

Peluang penelitian juga terlihat pada aspek evaluasi kinerja sistem yang direfaktor dengan metrik modern. Sebagian besar studi sebelumnya menyoroti pengukuran performa tanpa mengaitkannya secara langsung dengan hasil refaktorisasi arsitektur. Penelitian ini berpotensi memperluas pemahaman mengenai hubungan antara perbaikan struktur sistem dengan peningkatan metrik Core Web Vitals sebagai indikator objektif efisiensi aplikasi berbasis web.

Tabel 1. *Perbandingan Penelitian Terdahulu Terkait Refaktorisasi dan Kinerja Sistem*

No	Peneliti	Tahun	Fokus Penelitian	Hasil Utama
1	Zhou & Kim	2021	Pengelolaan <i>state</i> komponen	Reduksi kompleksitas

				dan redundansi data
2	Serrano et al.	2022	Perbandingan jQuery vs pendekatan deklaratif	Peningkatan konsistensi tampilan 23 %
3	Nugroho et al.	2022	Maintainability kode JavaScript	Penurunan <i>technical debt</i> 41 %
4	Rahman et al.	2023	Pendekatan utility-first CSS	Efisiensi pengembangan +47 %
5	Duarte et al.	2023	Pola arsitektur aplikasi web	Efisiensi pemeliharaan meningkat signifikan
6	Government Digital Service	2022	Dampak penghapusan jQuery di GOV.UK	Waktu interaktivitas ↑ 17 %
7	Gupta & Sharma	2024	Komparasi pustaka <i>state management</i>	Zustand lebih ringan dan efisien
8	Tan et al.	2022	Penerapan React Query berbasis cache	Permintaan jaringan ↓ 42 %

Tabel 2. Ringkasan Temuan Empiris Terkait Peningkatan Kinerja Sistem

Aspek yang Dianalisis	Teknologi atau Strategi	Dampak Utama	Sumber
<i>State Management</i>	React Query & Zustand	Sinkronisasi data lebih stabil, beban server turun	Tan et al., 2022; Gupta & Sharma, 2024
Modularitas Kode	Arsitektur berbasis komponen	Maintainability meningkat, redundansi meningkat	Rafi et al., 2022; Nugroho et al., 2022
Refaktorisasi Kode Warisan	Penghapusan jQuery	LCP dan TTI meningkat signifikan	WordPress Team, 2021; GOV.UK, 2022
Pengelolaan Gaya	Tailwind CSS (utility-first)	Konflik gaya menurun, kecepatan pengembangan meningkat	Rahman et al., 2023
Optimalisasi Berkas	Tree-shaking dan lazy-loading	FID lebih baik, memori klien menurun	Nakamura et al., 2023

2.2 Tinjauan Pustaka

2.2.1 Arsitektur Sistem Berbasis Komponen

Arsitektur frontend modern pada pengembangan aplikasi web mengadopsi pendekatan component-based architecture, yaitu metode pembangunan antarmuka yang memecah tampilan menjadi unit-unit kecil yang bersifat independen dan dapat digunakan kembali (reusable components). Menurut Rahman et al. (2023), pendekatan ini

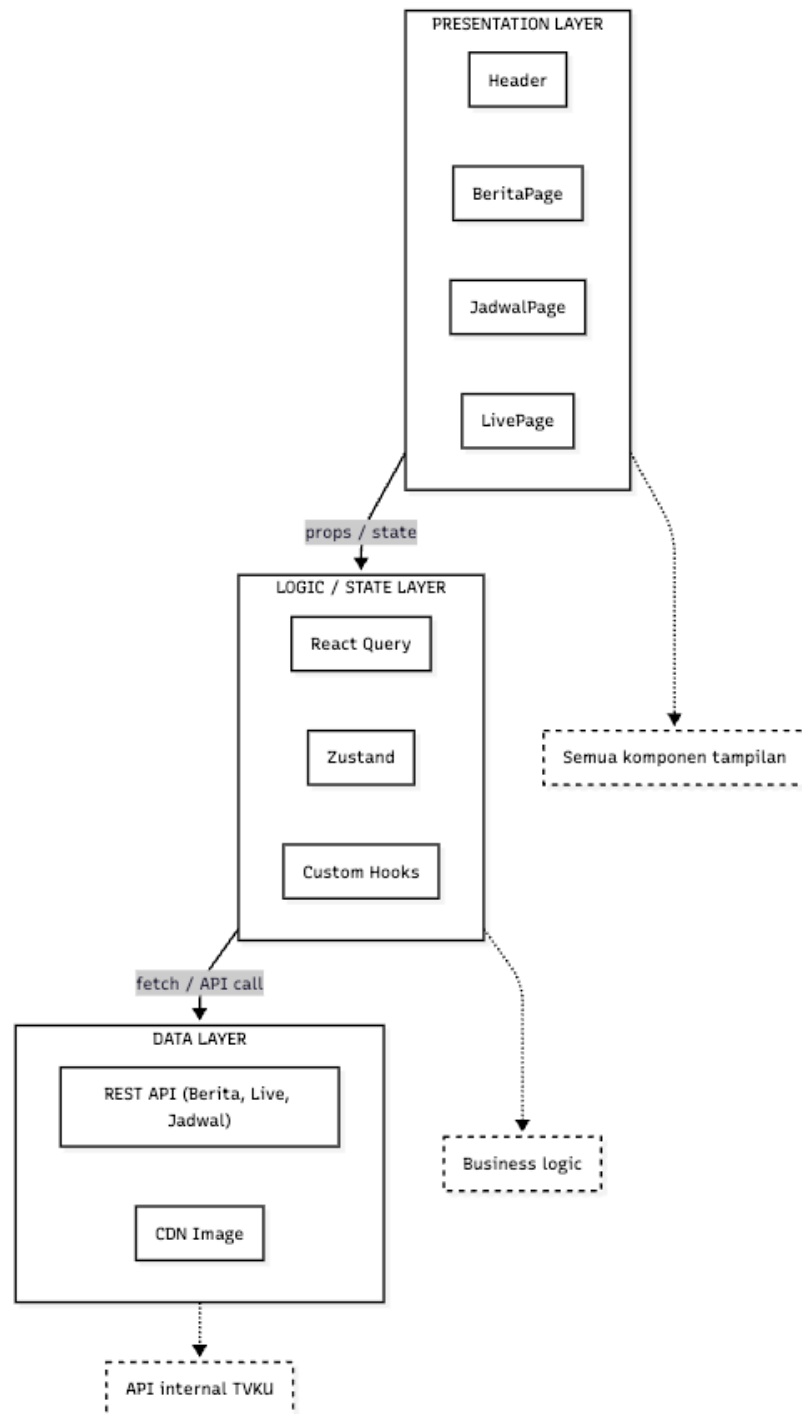
memungkinkan sistem dikembangkan secara modular, sehingga proses pemeliharaan dan ekspansi fitur menjadi lebih efisien. Setiap komponen memiliki logika, tampilan, serta dependensi tersendiri yang dapat diuji dan dioptimalkan tanpa memengaruhi bagian lain.

Dalam konteks pengembangan aplikasi TVKU, framework Next.js digunakan karena mendukung konsep server-side rendering (SSR), client-side rendering (CSR), dan incremental static regeneration (ISR) secara bersamaan. Menurut Tan et al. (2022), kombinasi ini memberikan fleksibilitas tinggi pada proses rendering, di mana data statis dan dinamis dapat diolah secara efisien untuk meningkatkan kecepatan pemuatan halaman. Pendekatan tersebut menjadikan sistem TVKU memiliki performa tinggi sekaligus mudah diperbarui secara real-time.

Secara umum, arsitektur berbasis komponen pada TVKU terdiri atas tiga lapisan utama:

1. Lapisan Presentasi (Presentation Layer), berisi komponen antarmuka seperti header, konten berita, dan player video yang berinteraksi langsung dengan pengguna.
2. Lapisan Logika Aplikasi (Logic Layer), berisi komponen yang mengatur pengambilan data, validasi, dan pengelolaan state.
3. Lapisan Data (Data Layer), terhubung dengan API eksternal untuk menyediakan data dinamis seperti berita, jadwal siaran, dan metadata video.

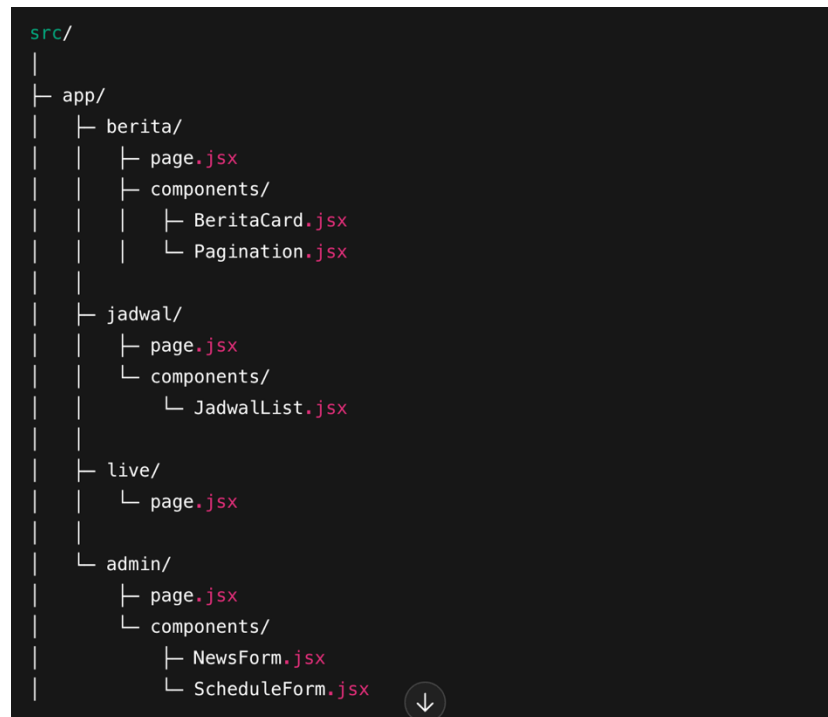
Menurut Ferreira et al. (2025), pemisahan tanggung jawab antar lapisan ini mendorong maintainability yang tinggi dan mengurangi risiko technical



Gambar 1. *Struktur Umum Arsitektur Berbasis Komponen Aplikasi TVKU*

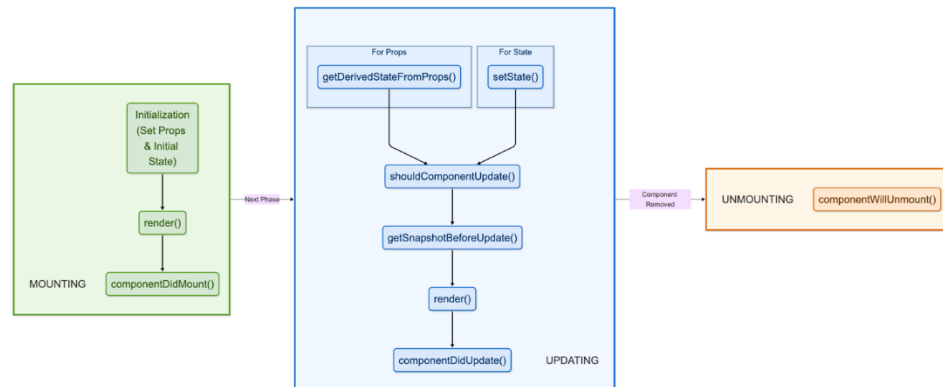
debt selama proses pengembangan. Setiap lapisan memiliki batas yang jelas melalui sistem props dan state sehingga komunikasi data lebih terkontrol.

Gambaran umum dari penerapan arsitektur berbasis komponen pada sistem TVKU dapat dilihat pada Gambar II.1, yang menampilkan relasi antar lapisan beserta alur komunikasi datanya.



Gambar 2. *Hierarki Modul Aplikasi TVKU*

Sementara itu, Gambar II.2 menjelaskan hierarki modul utama dalam frontend aplikasi, meliputi modul Berita, Jadwal, Live Streaming, dan Dashboard Admin. Setiap modul memiliki subkomponen yang dikembangkan secara independen untuk memastikan konsistensi dan kemudahan refaktorisasi.



Gambar 3. *Alur Lifecycle Rendering*

Proses rendering lifecycle pada komponen frontend juga menjadi bagian penting dari arsitektur ini. React dan Next.js menggunakan sistem virtual DOM untuk mengidentifikasi perubahan state dan melakukan pembaruan tampilan secara efisien. Menurut Booch et al. (2022), pendekatan ini menurunkan beban rendering pada browser karena hanya elemen yang berubah yang diperbarui. Gambar II.3 memperlihatkan alur lifecycle rendering komponen yang mencakup tahap mounting, updating, dan unmounting.

Dengan arsitektur seperti ini, sistem TVKU dapat dikembangkan secara bertahap melalui pendekatan scalable modular design. Setiap pengembang dapat bekerja pada modul yang berbeda tanpa mengganggu modul lain, dan seluruh sistem dapat diintegrasikan melalui kontrol state terpusat. Pendekatan ini tidak hanya meningkatkan efisiensi tim pengembang, tetapi juga menjaga kualitas kode tetap stabil seiring bertambahnya kompleksitas proyek.

2.2.2 Refaktorisasi Perangkat Lunak pada Frontend Modern

Refaktorisasi perangkat lunak merupakan proses restrukturisasi kode tanpa mengubah perilaku eksternal sistem, dengan tujuan meningkatkan kualitas internal, keterbacaan, serta efisiensi pemeliharaan. Menurut Ferreira et al. (2025), praktik refaktorisasi menjadi langkah krusial dalam menjaga stabilitas sistem yang berkembang cepat, terutama pada proyek

berbasis frontend modern yang melibatkan banyak komponen dan dependensi.

Dalam konteks pengembangan aplikasi berbasis React dan Next.js, refaktorisasi diperlukan untuk mengatasi code smell dan technical debt yang muncul akibat ekspansi fitur berulang tanpa pengelolaan struktur yang baik. Menurut Rahman et al. (2023), code smell mencakup pola kode seperti duplikasi fungsi, variabel tidak terpakai, dan logika bercabang berlebihan yang memperlambat proses debugging. Sementara itu, technical debt menggambarkan akumulasi konsekuensi teknis yang timbul karena keputusan pengembangan jangka pendek yang mengorbankan kualitas kode.

Proses refaktorisasi biasanya dilakukan melalui tahapan sistematis:

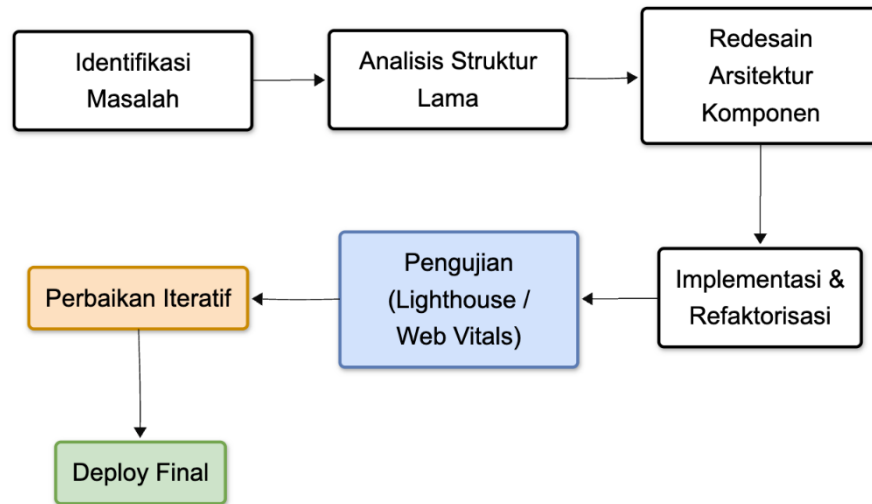
1. Identifikasi Masalah Kode, mencakup analisis kompleksitas fungsi dan deteksi duplikasi.
2. Perancangan Ulang Struktur, memecah komponen besar menjadi unit kecil yang lebih modular.
3. Pengujian Ulang Fungsionalitas, memastikan tidak terjadi perubahan perilaku eksternal.
4. Penerapan Otomasi, seperti integrasi linting dan code formatting untuk menjaga konsistensi.

Menurut Nyirongo et al. (2024), penerapan refaktorisasi pada proyek berbasis frontend juga harus mempertimbangkan efisiensi build time dan bundle size. Dengan membagi kode berdasarkan tanggung jawab dan menghapus dependensi usang, sistem dapat lebih cepat dimuat dan mudah dikembangkan.

Tabel 3. *Jenis dan Tujuan Refaktorisasi dalam Rekayasa Perangkat Lunak*

Jenis Refaktorisasi	Tujuan Utama	Dampak terhadap Sistem	Sumber
Code Clean-up	Menghapus redundansi kode dan duplikasi fungsi	Peningkatan keterbacaan kode	Ferreira et al. (2025)
Modularization	Memisahkan fungsi berdasarkan tanggung jawab	Maintainability meningkat	Nyirongo et al. (2024)
Dependency Removal	Menghapus pustaka usang seperti jQuery	Ukuran berkas naik, performa turun	GOV.UK (2022)
State Refactoring	Memusatkan logika pengelolaan data	Keterlacakan data meningkat	Zhou & Kim (2021)
Styling Refactoring	Menerapkan pendekatan utility-first	Menerapkan pendekatan utility-first	Rahman et al. (2023)

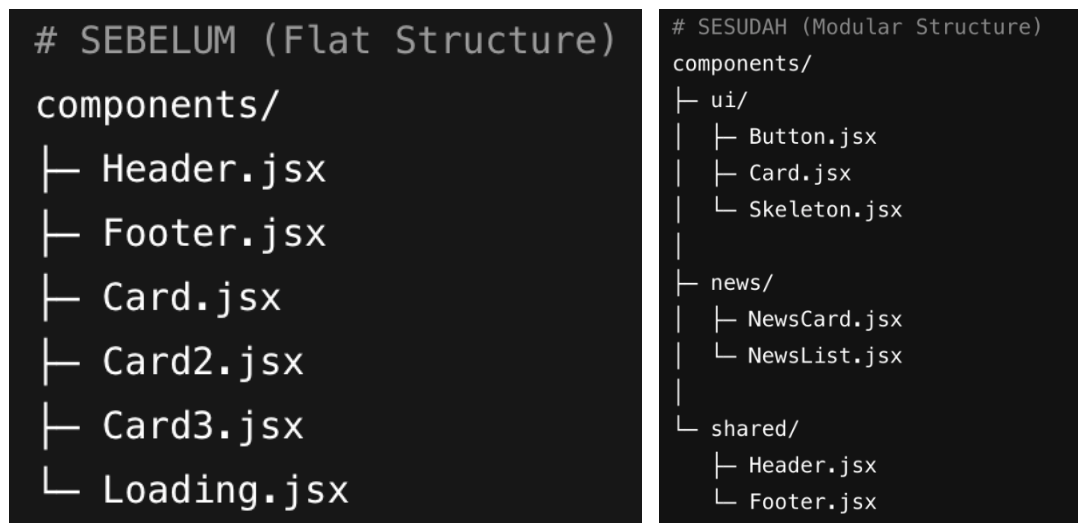
Menurut GOV.UK (2022), strategi refaktorisasi yang efektif tidak hanya memperbaiki kode, tetapi juga menyederhanakan arsitektur proyek secara menyeluruh. Dalam sistem TVKU, refaktorisasi dilakukan untuk



Gambar 4. Alur Proses Refaktorisasi Clinet Site TVKU

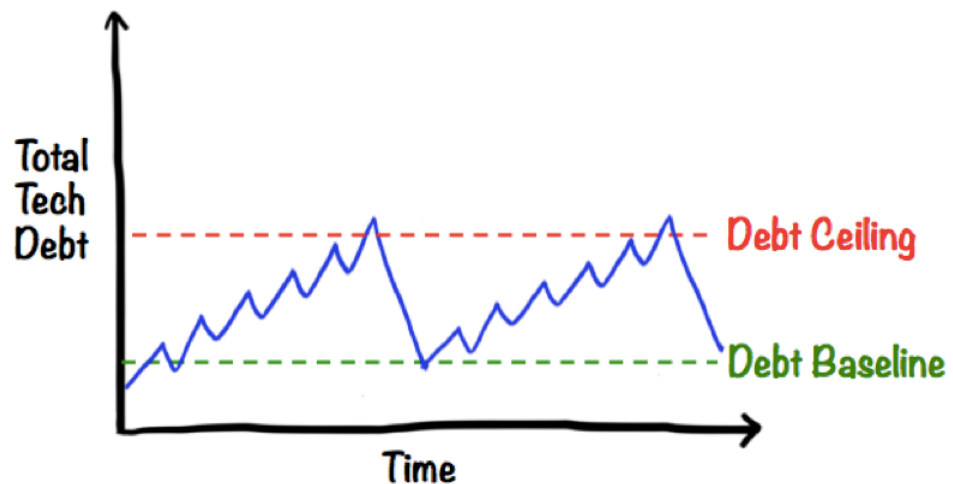
memisahkan logika state management, menghapus pustaka lama, serta menata ulang struktur folder agar selaras dengan prinsip component-driven development.

Gambar II.4 menampilkan alur proses refaktorisasi perangkat lunak pada frontend TVKU, mulai dari tahap identifikasi masalah hingga implementasi kode baru yang sudah dioptimalkan. Sedangkan Gambar II.5 memperlihatkan perbandingan struktur komponen sebelum dan sesudah refaktorisasi, di mana hierarki baru menjadi lebih modular, terukur, dan mudah diperluas.



Gambar 5. Struktur Komponen Sebelum & Setelah Refaktorisasi

Selain meningkatkan performa, refaktorisasi juga berperan penting dalam mitigasi risiko technical debt. Menurut Zhou dan Kim (2021), hubungan antara code smell, technical debt, dan refaktorisasi bersifat siklikal: peningkatan kompleksitas kode menyebabkan hutang teknis, yang kemudian diatasi dengan refaktorisasi, dan hasilnya kembali menjaga kualitas sistem. Hubungan ini divisualisasikan pada Gambar 6.



Gambar 6. Code Smell – Technical Debt – Refactoring

Sumber: <https://blog.crisp.se/2013/10/11/henrikkniberg/good-and-bad-technical-debt>

Penerapan refaktorisasi secara berkelanjutan mendorong sistem menjadi lebih stabil dan efisien. Pada proyek TVKU, refaktorisasi bukan hanya langkah perbaikan, tetapi juga strategi evolusi sistem yang berkelanjutan, memastikan performa aplikasi tetap optimal seiring penambahan fitur dan kompleksitas.

2.2.3 Pola Aliran dan Manajemen Data Frontend

Dalam pengembangan aplikasi berbasis komponen, pengelolaan data memiliki peran sentral untuk memastikan konsistensi informasi antar komponen serta efisiensi proses rendering. Menurut Gupta dan Sharma (2024), tantangan utama pada sistem frontend modern adalah bagaimana data global dan lokal dikelola secara sinkron tanpa menurunkan performa. Oleh karena itu, pola aliran data dan state management menjadi elemen kunci dalam menjaga stabilitas antarmuka.

Secara umum, terdapat dua pendekatan utama dalam manajemen data frontend: state management global dan local component state. Pendekatan global digunakan ketika banyak komponen berbagi data yang sama, sedangkan local state lebih efisien untuk data spesifik yang hanya relevan dalam lingkup satu komponen. React sebagai library inti menyediakan `useState` dan `useContext` untuk pengelolaan sederhana, namun pada aplikasi berskala besar seperti TVKU, dibutuhkan manajemen data tingkat lanjut menggunakan pustaka tambahan seperti Redux, React Query, dan Zustand.

Redux merupakan pustaka manajemen state global berbasis konsep store terpusat. Menurut Tan et al. (2022), Redux memberikan kontrol ketat terhadap aliran data satu arah (unidirectional data flow), sehingga setiap perubahan state dapat dilacak dengan mudah. Meskipun stabil, pendekatan ini memerlukan konfigurasi panjang dan boilerplate code yang tinggi.

Berbeda dengan Redux, React Query berfokus pada sinkronisasi data eksternal melalui API. Menurut Gupta dan Sharma (2024), React Query secara otomatis menangani proses fetching, caching, dan background synchronization, sehingga data dari server selalu mutakhir tanpa perlu

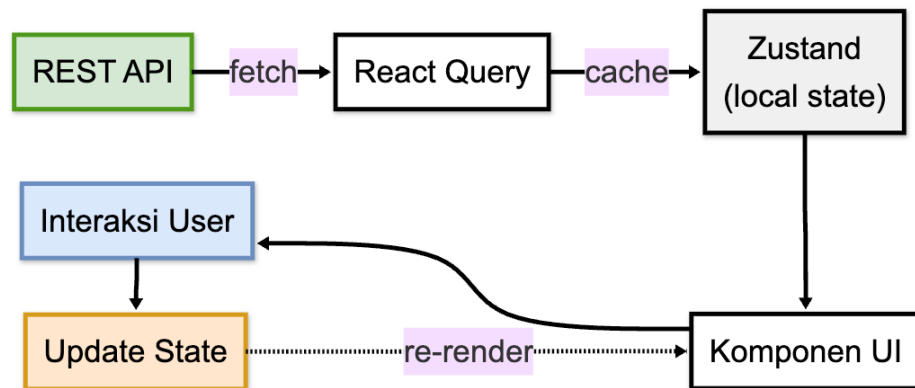
intervensi manual. Pendekatan ini meningkatkan efisiensi untuk data dinamis seperti berita dan jadwal siaran pada sistem TVKU.

Sementara itu, Zustand menawarkan alternatif ringan dengan konsep client state management berbasis hooks. Menurut Rahman et al. (2023), pustaka ini cocok untuk sistem menengah karena sintaksnya sederhana dan mendukung pembagian state secara modular. Dengan kombinasi ini, pengembang dapat memilih strategi pengelolaan data yang paling sesuai dengan kompleksitas fitur.)

Tabel 4. *Perbandingan Teknologi Pengelolaan Data Modern*

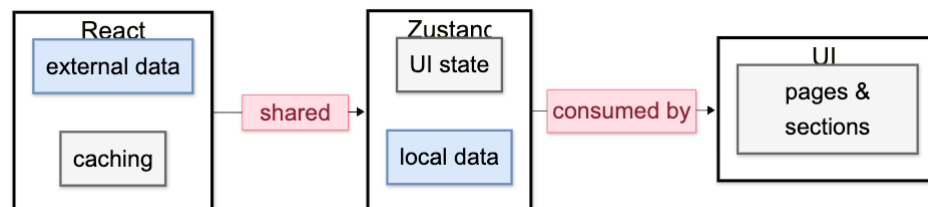
Aspek	Redux	React Query	Zustand
Pendekatan	Terpusat (global state)	Sinkronisasi data eksternal	Client state ringan
Konfigurasi	Kompleks (boilerplate tinggi)	Otomatis melalui hook	Sederhana dan modular
Caching	Tidak tersedia	Mendukung smart caching	Tidak prioritas
Integrasi API	Manual	Otomatis (async-aware)	Dapat dikombinasikan
Performa	Stabil untuk sistem besar	Efisien untuk data dinamis	Ringan untuk sistem menengah
Referensi	Gupta & Sharma (2024)	Tan et al. (2022)	Gupta & Sharma (2024)

Pada implementasi sistem TVKU, kombinasi React Query untuk data eksternal dan Zustand untuk local state terbukti paling efisien. Aliran data digambarkan secara hierarkis dalam Gambar II.7, di mana data dikirim dari API eksternal melalui fetch layer menuju state global sebelum didistribusikan ke setiap komponen UI.



Gambar 7. Pola Aliran Data dalam Aplikasi Berbasis Komponen

Selanjutnya, Gambar II.8 memperlihatkan arsitektur konseptual manajemen data frontend TVKU, yang menunjukkan bagaimana tiap modul (Berita, Jadwal, Live, Dashboard) terhubung dengan lapisan pengelolaan data.



Gambar 8. Arsitektur Konseptual Manajemen Data Client Site TVKU

Menurut Ferreira et al. (2025), pendekatan hybrid seperti ini meningkatkan traceability dan mengurangi potensi state conflict antar komponen. Selain itu, dengan refaktorisasi sistem penyimpanan data menggunakan pola modular, dependensi antar komponen dapat dikurangi secara signifikan, yang berdampak langsung terhadap peningkatan performa rendering. Secara keseluruhan, manajemen data yang efisien memungkinkan proses pengembangan frontend menjadi lebih terukur dan adaptif terhadap perubahan. Struktur aliran data yang baik tidak hanya meningkatkan

kecepatan interaksi pengguna, tetapi juga menjadi fondasi penting dalam mendukung skalabilitas sistem secara keseluruhan.

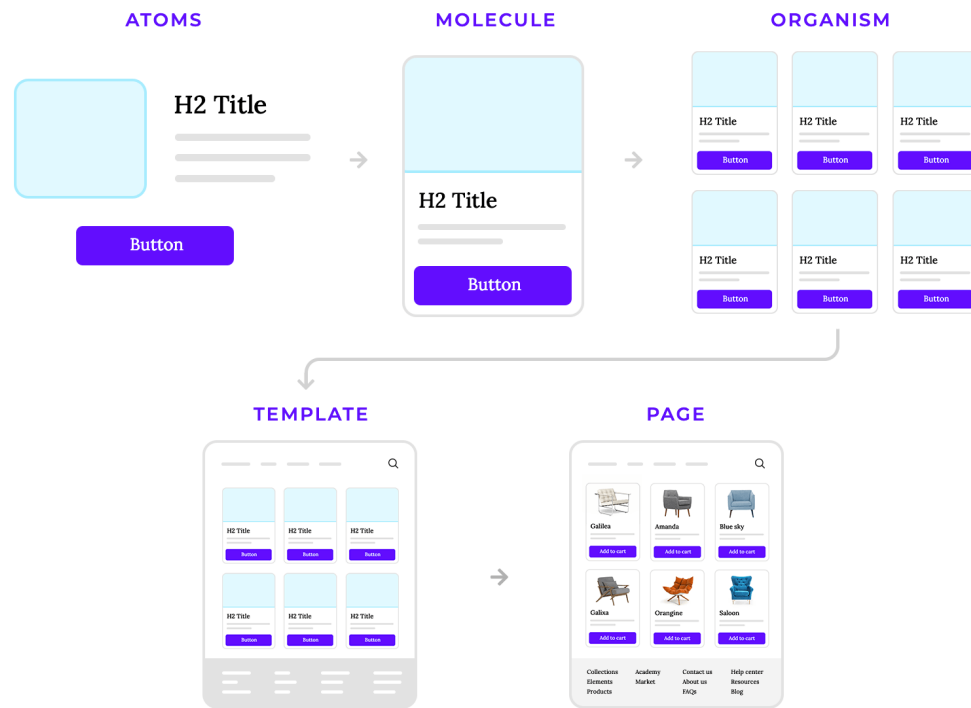
2.2.4 Desain UI Modern Berbasis Utility-First

Desain antarmuka modern menuntut sistem yang efisien, konsisten, dan mudah disesuaikan dengan kebutuhan pengguna. Salah satu pendekatan yang kini populer dalam pengembangan frontend adalah utility-first CSS framework, di mana setiap gaya visual direpresentasikan melalui kelas utilitas kecil yang spesifik. Menurut Rahman et al. (2023), pendekatan ini memungkinkan proses styling dilakukan secara modular dan terukur, karena setiap komponen hanya menggunakan kelas yang dibutuhkan tanpa harus menulis ulang kode CSS panjang.

Framework Tailwind CSS merupakan implementasi paling representatif dari konsep ini. Tailwind menyediakan kumpulan kelas utilitas yang mencakup kontrol terhadap layout, warna, tipografi, hingga animasi. Menurut Tan et al. (2022), penggunaan Tailwind mampu mempercepat proses pengembangan UI hingga 30% karena mengurangi waktu styling manual dan menghindari konflik antar file CSS. Selain itu, Tailwind mendukung strategi purge dan JIT (Just-In-Time) compilation, yang secara otomatis menghapus kelas tak terpakai dari bundle akhir sehingga ukuran berkas menjadi sangat ringan.

Pendekatan desain yang digunakan dalam sistem TVKU mengombinasikan dua konsep utama: Utility-First CSS dan Atomic Design System. Atomic Design menekankan pembentukan elemen UI dari unit paling kecil (atoms) hingga komponen kompleks (organisms dan templates). Menurut Ferreira et al. (2025), kombinasi ini memudahkan pengembang menjaga konsistensi visual sekaligus memungkinkan reusabilitas tinggi di seluruh modul aplikasi.

Gambar II.9 menggambarkan hierarki atomic design system yang diadaptasi dalam proyek TVKU, sedangkan Tabel II.5 menampilkan perbandingan beberapa framework CSS populer untuk menunjukkan keunggulan Tailwind.)



Gambar 9. Atomic design hierarchy diagram

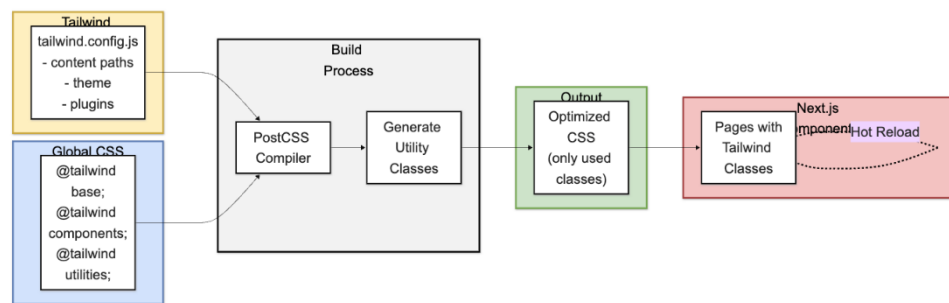
Sumber: <https://www.justinmind.com/ui-design/atomic-design>

Tabel 5. Perbandingan Framework CSS Modern

Aspek	Bootstrap 5	Tailwind CSS	Chakra UI
Pendekatan	Komponen siap pakai	Utility-first modular	Component-based library
Kustomisasi	Terbatas (variabel SCSS)	Tinggi (kelas bebas)	Sedang
Ukuran bundle	±160 KB	±70 KB (setelah purge)	±150 KB

Adaptasi mobile	Responsif otomatis	Kelas responsif spesifik	Responsif adaptif
Performa build	Sedang	Sangat cepat (JIT)	Sedang
Referensi	Ferreira et al. (2025)	Rahman et al. (2023)	Tan et al. (2022)

Pipeline kerja Tailwind pada sistem TVKU meliputi tahap kompilasi file konfigurasi, pembentukan kelas dinamis berbasis utility, dan integrasi otomatis ke dalam komponen Next.js. Gambar II.10 menunjukkan alur pipeline proses Tailwind CSS, di mana setiap perubahan pada file konfigurasi langsung tercermin dalam tampilan komponen melalui hot reload system.



Gambar 10. *Tailwind Css Pipeline Diagram*

Menurut Nyirongo et al. (2024), pendekatan ini tidak hanya mempercepat iterasi pengembangan tetapi juga mendukung design consistency enforcement, yaitu mekanisme di mana seluruh elemen UI mengacu pada satu sistem konfigurasi warna dan tipografi yang sama. Dengan demikian, penerapan utility-first di TVKU tidak hanya memudahkan refaktorisasi antarmuka, tetapi juga meningkatkan maintainability jangka panjang.

Kombinasi Tailwind CSS dengan prinsip atomic design terbukti efektif dalam pengembangan sistem frontend berskala menengah seperti TVKU. Pendekatan ini memberikan keseimbangan antara fleksibilitas desain dan

efisiensi pengembangan, memungkinkan tim untuk berfokus pada pengalaman pengguna tanpa terbebani oleh kompleksitas styling manual.

2.2.5 Evaluasi dan Optimalisasi Kinerja Sistem Web

Kinerja (performance) merupakan indikator utama dalam menilai kualitas sistem web modern. Menurut Gupta dan Sharma (2024), kinerja aplikasi web tidak hanya dipengaruhi oleh kecepatan pemuatan halaman, tetapi juga oleh stabilitas interaksi pengguna dan efisiensi pemrosesan sumber daya. Evaluasi kinerja diperlukan untuk memastikan bahwa pengalaman pengguna tetap optimal di berbagai perangkat dan kondisi jaringan.

Salah satu metode evaluasi yang banyak digunakan adalah Core Web Vitals yang diperkenalkan oleh Google. Menurut Tan et al. (2022), metrik ini mengukur tiga aspek utama: Largest Contentful Paint (LCP), First Input Delay (FID), dan Cumulative Layout Shift (CLS).

- LCP menggambarkan waktu yang dibutuhkan untuk memuat elemen terbesar pada tampilan utama.
- FID menilai kecepatan sistem dalam merespons interaksi pertama pengguna.
- CLS mengukur stabilitas tata letak visual selama proses pemuatan.

Tabel 6. *Nilai Ideal Core Web Vitals*

Metrik	Kategori Baik	Perlu Ditingkatkan	Buruk
LCP	≤ 2.5 detik	2.5–4.0 detik	> 4.0 detik
FID	≤ 100 ms	100–300 ms	> 300 ms
CLS	≤ 0.1	0.1–0.25	> 0.25

Evaluasi terhadap sistem TVKU dilakukan dengan menganalisis loading time, render blocking resources, dan runtime execution. Menurut Ferreira et al. (2025), performa frontend dapat ditingkatkan melalui pengelolaan sumber daya secara efisien, termasuk pemangkasan

dependensi, pengurangan bundle size, dan pemanfaatan teknik lazy loading untuk komponen besar seperti video player.

Beberapa strategi optimasi yang diterapkan pada sistem web modern antara lain:

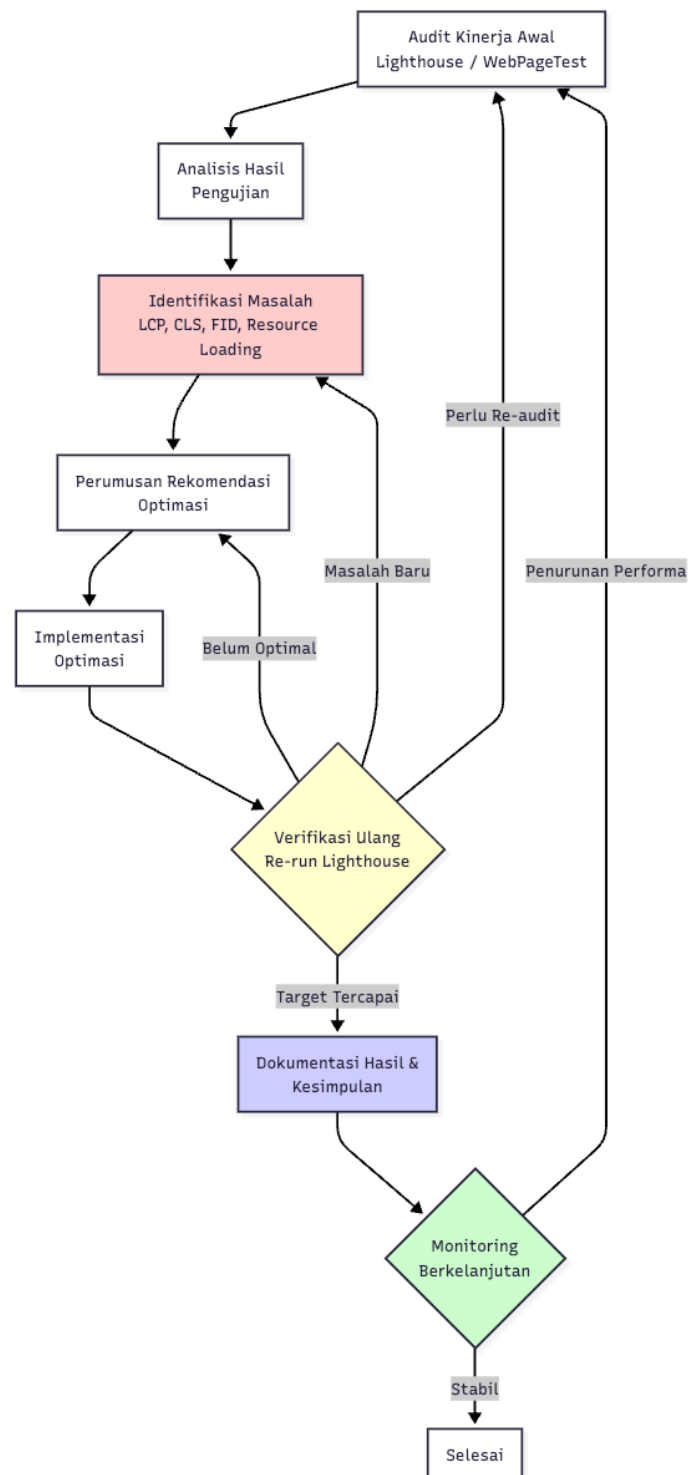
1. Code Splitting – membagi kode aplikasi ke dalam beberapa bagian agar hanya modul yang dibutuhkan yang dimuat.
2. Tree Shaking – menghapus fungsi atau modul yang tidak digunakan dalam build akhir.
3. Image Optimization – menggunakan format modern seperti WebP untuk mengurangi ukuran file.
4. Server-Side Rendering (SSR) – mempercepat first paint dengan memuat halaman dari server.
5. Caching dan CDN – memanfaatkan distribusi global konten untuk mempercepat pengiriman data.

Tabel 7. Teknik Optimasi Kinerja Frontend Modern

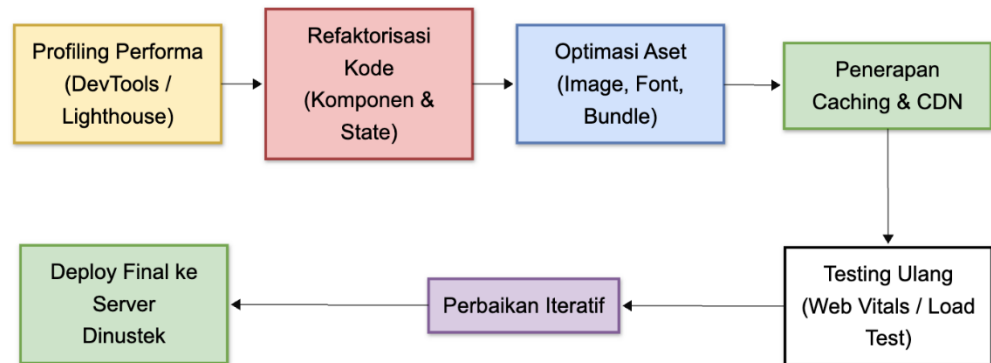
Teknik Optimasi	Tujuan	Dampak Utama
Code Splitting	Mengurangi waktu pemuatan awal	Waktu <i>initial load</i> lebih cepat
Tree Shaking	Menghapus modul tidak terpakai	Ukuran bundle berkurang
Image Optimization	Mengurangi beban jaringan	LCP meningkat signifikan
SSR	Mempercepat rendering awal	Responsivitas halaman meningkat
Caching & CDN	Menyediakan data dari lokasi terdekat	Latency menurun drastis

Gambar II.11 menggambarkan proses umum evaluasi performa sistem web yang mencakup tahap analisis, pengujian, dan interpretasi hasil. Setelah evaluasi, langkah optimasi dilakukan untuk memperbaiki bagian yang menjadi bottleneck. Sementara itu, Gambar II.12 memperlihatkan pipeline

proses optimalisasi frontend TVKU, mulai dari deteksi elemen berat hingga hasil pengujian ulang setelah perbaikan.



Gambar 11. *Proses Evaluasi Kinerja Web Modern*



Gambar 12. *Optimalisasi Performa Frontend*

Menurut Rahman et al. (2023), optimalisasi yang berkelanjutan dapat mengurangi waktu muat hingga 40% pada aplikasi berbasis React. Selain itu, penerapan performance budget (yaitu batas maksimum ukuran sumber daya tiap halaman) dapat mencegah penurunan performa saat sistem berkembang. Pendekatan ini menjadi bagian penting dalam refaktorisasi berkelanjutan yang diterapkan pada sistem TVKU.

Dengan strategi evaluasi dan optimasi yang tepat, sistem web tidak hanya menjadi lebih cepat, tetapi juga lebih efisien dalam penggunaan sumber daya. Proses ini membentuk siklus pengembangan berkelanjutan yang mendukung pengalaman pengguna yang stabil dan responsif.

2.2.6 Aksesibilitas dan Pengalaman Pengguna (UX)

Aksesibilitas dan pengalaman pengguna (User Experience/UX) merupakan dua aspek penting dalam perancangan sistem web modern yang berorientasi pada pengguna. Menurut Ferreira et al. (2025), sistem yang baik tidak hanya berfungsi secara teknis, tetapi juga mampu diakses dan digunakan oleh berbagai kalangan, termasuk pengguna dengan keterbatasan visual, motorik, maupun kognitif. Penerapan prinsip aksesibilitas dalam pengembangan antarmuka web bertujuan memastikan bahwa seluruh pengguna dapat berinteraksi dengan sistem secara efektif, efisien, dan menyenangkan.

Standar utama yang digunakan sebagai acuan global adalah Web Content Accessibility Guidelines (WCAG) versi 2.2 yang dikembangkan oleh W3C. Menurut Gupta dan Sharma (2024), WCAG menetapkan empat prinsip dasar yang harus dipenuhi, yaitu:

1. Perceivable – informasi dan komponen antarmuka harus dapat dirasakan oleh semua pengguna.
2. Operable – elemen sistem harus dapat dioperasikan menggunakan berbagai metode input.
3. Understandable – isi dan perilaku sistem harus mudah dipahami.
4. Robust – konten harus kompatibel dengan berbagai teknologi asisten seperti screen reader.

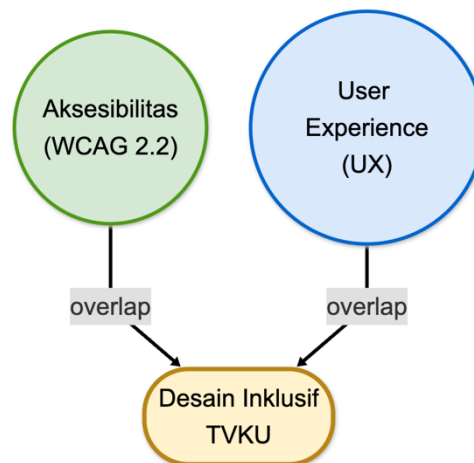
Integrasi prinsip-prinsip tersebut pada sistem TVKU diwujudkan melalui desain antarmuka yang kontras, navigasi berbasis keyboard, serta dukungan teks alternatif pada elemen gambar dan video. Tabel II.8 berikut menampilkan perbandingan elemen aksesibilitas dan kontribusinya terhadap peningkatan UX.

Tabel 8. *Perbandingan Elemen Aksesibilitas dan Pengalaman Pengguna (UX)*

Elemen	Fokus Aksesibilitas	Dampak terhadap UX
Kontras Warna	Meningkatkan keterbacaan teks	Mengurangi kelelahan visual pengguna
Navigasi Keyboard	Akses bagi pengguna non-mouse	Meningkatkan efisiensi interaksi
Teks Alternatif	Membantu pengguna dengan gangguan visual	Memperluas pemahaman konten
Struktur Heading	Membantu pembacaan oleh <i>screen reader</i>	Menyusun informasi secara hierarkis
Validasi Form	Mengarahkan pengguna ke kesalahan input	Mengurangi frustrasi pengguna

Menurut Rahman et al. (2023), integrasi UX yang efektif tidak hanya memperhatikan tampilan antarmuka, tetapi juga mencakup aspek emosional pengguna seperti kenyamanan, kejelasan, dan rasa kontrol. Pendekatan human-centered design memastikan bahwa sistem beradaptasi dengan perilaku pengguna, bukan sebaliknya.

Dalam sistem TVKU, penerapan UX difokuskan pada dua hal: konsistensi visual dan kesederhanaan interaksi. Komponen UI dirancang dengan gaya minimalis dan responsif menggunakan Tailwind CSS, sementara logika interaksi dibuat seragam antar modul untuk mengurangi beban kognitif pengguna.



Gambar 13. Hubungan antara Aksesibilitas dan UX dalam Sistem Frontend TVKU

Gambar 13 menggambarkan hubungan antara aksesibilitas dan UX dalam sistem frontend TVKU. Aksesibilitas menjadi fondasi dasar yang memastikan keterjangkauan sistem, sedangkan UX menjadi lapisan adaptif yang meningkatkan kepuasan pengguna secara keseluruhan. Kombinasi keduanya menciptakan pengalaman digital yang inklusif dan berkelanjutan. Menurut Nyirongo et al. (2024), penerapan aksesibilitas dan UX yang terintegrasi juga memberikan keuntungan bagi keberlanjutan sistem jangka panjang. Aplikasi yang ramah pengguna cenderung memiliki tingkat user retention lebih tinggi serta meminimalkan biaya perawatan akibat keluhan pengguna. Oleh karena itu, strategi desain yang diadopsi TVKU menjadi

bagian dari pendekatan rekayasa perangkat lunak yang tidak hanya berorientasi pada teknologi, tetapi juga pada nilai kemanusiaan.

2.2.7 UML dalam Analisis dan Perancangan Sistem

Unified Modeling Language (UML) merupakan standar internasional untuk memodelkan, memvisualisasikan, dan mendokumentasikan sistem perangkat lunak. Menurut Booch et al. (2022), UML menyediakan seperangkat diagram yang berfungsi menggambarkan struktur, perilaku, serta interaksi antar komponen sistem secara konsisten. Pendekatan ini penting dalam pengembangan perangkat lunak berbasis objek karena membantu pengembang memahami alur kerja sistem sebelum implementasi dilakukan.

Dalam konteks rekayasa perangkat lunak modern, UML digunakan sebagai alat bantu analisis dan komunikasi antara pengembang, perancang antarmuka, serta pihak pemangku kepentingan. Menurut Rahman et al. (2023), penggunaan UML dapat mempercepat proses desain karena memungkinkan pengembang mengidentifikasi kesalahan logika sejak tahap perancangan, bukan setelah implementasi kode.

Sistem frontend TVKU menggunakan beberapa diagram UML utama untuk menggambarkan hubungan antar komponen dan proses sistem, meliputi:

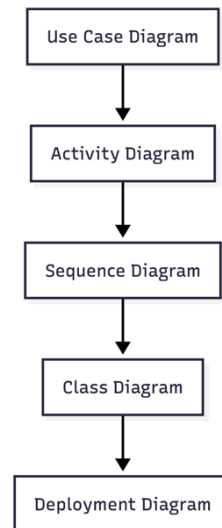
1. Use Case Diagram, menjelaskan interaksi pengguna dengan sistem dan batas fungsionalitas aplikasi.
2. Activity Diagram, menggambarkan alur kerja dan logika proses seperti manajemen konten berita dan pengaturan jadwal siaran.
3. Sequence Diagram, memperlihatkan urutan komunikasi antar komponen frontend dan API backend.
4. Class Diagram, menampilkan struktur kelas, atribut, dan relasi antarkomponen React.
5. Deployment Diagram, menunjukkan distribusi modul frontend dan server dalam infrastruktur Next.js.

Tabel 9. *Jenis dan Konteks Penggunaan Diagram UML pada Sistem TVKU*

Jenis Diagram	Fungsi Utama	Konteks Penggunaan di TVKU
Use Case Diagram	Menunjukkan aktor dan interaksi sistem	Menggambarkan peran admin dan pengguna publik
Activity Diagram	Menjelaskan alur aktivitas sistem	Menggambarkan proses manajemen konten berita
Sequence Diagram	Memvisualisasikan urutan pesan	Menggambarkan alur komunikasi frontend-API
Class Diagram	Menunjukkan struktur dan atribut kelas	Mewakili komponen React dan dependensinya
Deployment Diagram	Menunjukkan distribusi komponen	Menjelaskan arsitektur Next.js dan server Vercel

Menurut Ferreira et al. (2025), diagram UML membantu menjaga keselarasan antara desain konseptual dan implementasi aktual. Dalam sistem TVKU, UML digunakan sebagai panduan pengembangan agar seluruh tim memiliki pemahaman yang sama mengenai struktur aplikasi dan dependensi yang terlibat.

Gambar II.14 memperlihatkan hubungan antar diagram UML dalam sistem frontend TVKU. Diagram tersebut menampilkan keterkaitan logis antara use case, activity, dan sequence diagram yang menjadi pondasi bagi perancangan kelas serta distribusi komponen pada tahap implementasi.

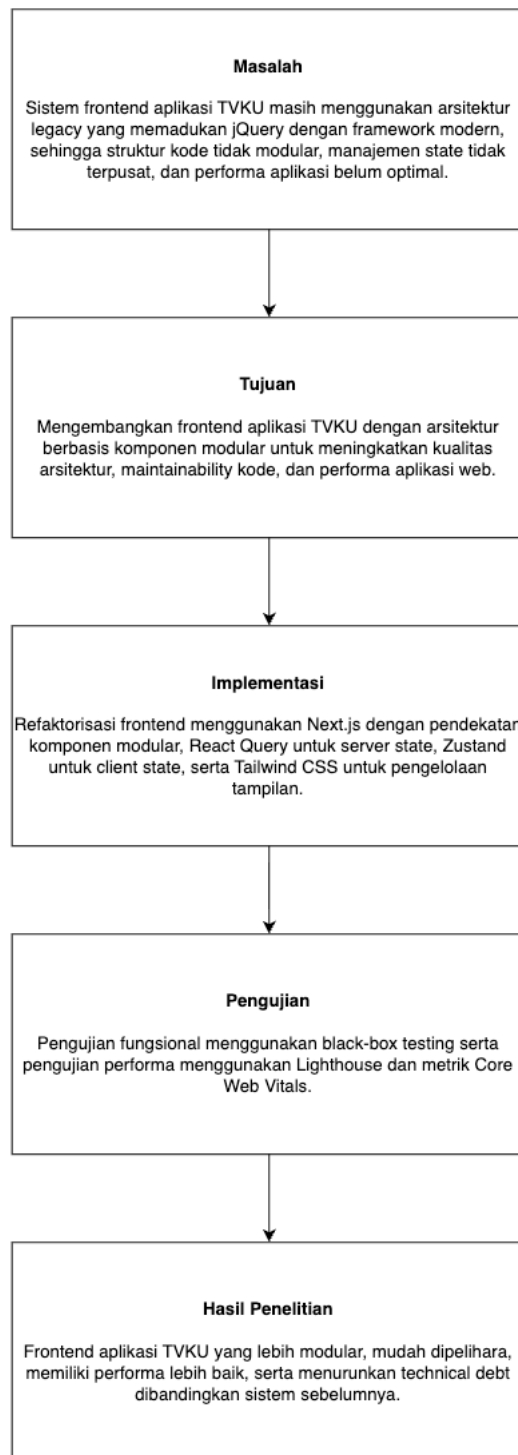


Gambar 14. *Hubungan Antar Digram UML*

Penerapan UML dalam proyek ini juga mendukung prinsip model-driven development (MDD), di mana desain sistem menjadi dasar langsung bagi proses implementasi. Menurut Nyirongo et al. (2024), MDD meminimalkan risiko kesalahan desain sekaligus meningkatkan efisiensi kolaborasi antar pengembang karena semua keputusan teknis sudah terdokumentasi dengan jelas sejak tahap awal.

Dengan demikian, penggunaan UML pada sistem TVKU bukan hanya sebagai alat visualisasi, tetapi juga sebagai komponen strategis dalam menjaga konsistensi, kualitas, dan skalabilitas pengembangan perangkat lunak.

2.3 Kerangka Pemikiran



Gambar 15. Kerangka Pemikiran

BAB III

METODOLOGI PENELITIAN

3.1 Metode Pengumpulan Data

3.1.1 Observasi Langsung

Observasi dilakukan secara langsung terhadap sistem frontend TVKU yang telah berjalan sebelumnya untuk mengidentifikasi struktur arsitektur, komponen antarmuka, serta alur data yang digunakan. Menurut Nyirongo et al. (2024), metode observasi efektif digunakan untuk memahami kondisi aktual sistem sebelum dilakukan perubahan atau optimasi.

Dalam tahap ini, peneliti mencatat berbagai temuan seperti:

1. Penggunaan pustaka frontend yang masih bersifat statis dan tidak modular.
2. Struktur folder dan file yang belum menerapkan component-based architecture secara konsisten.
3. Kinerja loading time halaman yang belum optimal berdasarkan hasil pengujian Lighthouse dan PageSpeed Insights.

Data hasil observasi ini menjadi dasar dalam proses analisis dan perancangan ulang sistem menggunakan pendekatan refaktorisasi perangkat lunak sebagaimana dijelaskan pada Bab II.

3.1.2 Wawancara Terbatas

Metode wawancara dilakukan secara semi-terstruktur dengan tim pengelola situs TVKU dan beberapa pengguna internal (staf redaksi). Menurut Gupta dan Sharma (2024), wawancara semi-terstruktur memungkinkan peneliti memperoleh informasi mendalam tanpa mengikat responden pada format pertanyaan kaku.

Pertanyaan wawancara difokuskan pada aspek:

1. Permasalahan teknis yang sering muncul pada sistem sebelumnya.

2. Harapan pengguna terhadap peningkatan performa dan kemudahan pengelolaan konten.
3. Proses operasional harian dalam mengelola berita dan jadwal siaran.

Hasil wawancara kemudian digunakan untuk menyesuaikan rancangan sistem baru agar lebih sesuai dengan kebutuhan pengguna aktual.

3.1.3 Studi Pustaka

Studi pustaka dilakukan untuk memperkuat dasar teori dan membandingkan pendekatan pengembangan sistem yang relevan. Menurut Rahman et al. (2023), studi pustaka penting dalam penelitian terapan karena menjadi landasan dalam memilih metode dan teknologi yang tepat.

Sumber literatur yang digunakan meliputi:

1. Buku dan jurnal ilmiah tentang rekayasa perangkat lunak, refaktorisasi kode, dan pengembangan frontend modern.
2. Dokumentasi resmi framework Next.js, React, dan Tailwind CSS.
3. Artikel teknis dan white paper terkait pengukuran performa sistem berbasis Core Web Vitals (Google, 2024).

Seluruh sumber pustaka digunakan untuk memastikan bahwa setiap keputusan teknis dalam pengembangan sistem memiliki dasar teoretis yang kuat dan relevan dengan perkembangan teknologi terkini.

3.1.4 Hasil Pengumpulan Data

Hasil pengumpulan data diperoleh melalui observasi langsung, wawancara terbatas, serta telaah literatur yang relevan. Secara umum ditemukan beberapa poin utama sebagai dasar penyusunan rancangan sistem.

Dari observasi, sistem frontend yang berjalan saat ini menunjukkan inkonsistensi struktur komponen, ketergantungan pada teknologi lama, serta waktu muat halaman yang belum optimal. Wawancara dengan tim internal mengindikasikan kebutuhan akan antarmuka yang lebih stabil,

responsif, dan mudah dipelihara. Sementara itu, studi pustaka menguatkan bahwa penerapan arsitektur modern dan pengurangan technical debt dapat meningkatkan performa serta keberlanjutan pengembangan.

Secara keseluruhan, temuan ini menegaskan perlunya pembaruan pada arsitektur frontend sebagai langkah untuk memperbaiki efisiensi, stabilitas, dan pengalaman pengguna.

3.2 Metode yang diusulkan

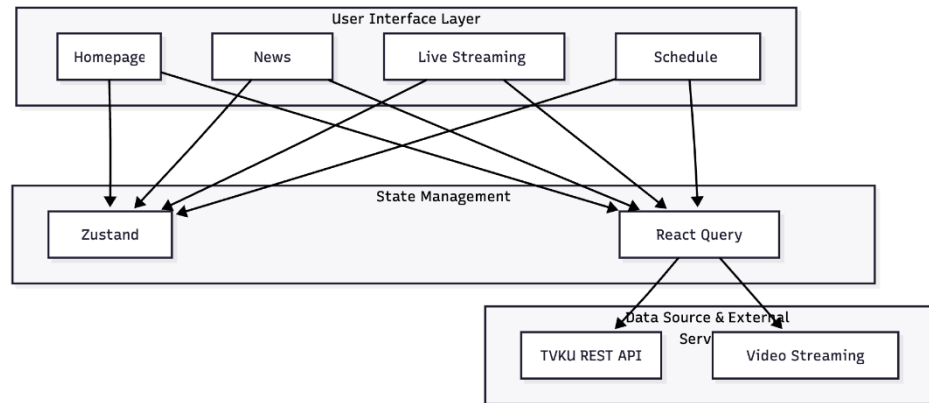
3.2.1 Deskripsi Umum Sistem

Sistem TVKU terdiri atas empat modul utama yang saling terintegrasi:

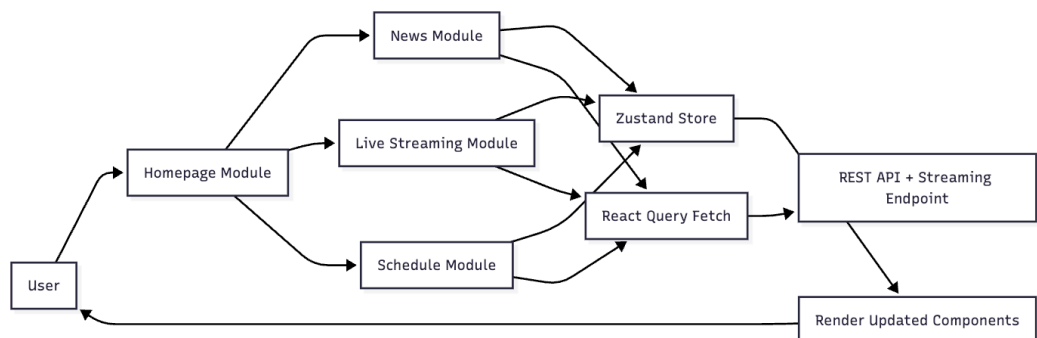
1. Modul Berita, menampilkan daftar dan detail berita terkini dari database internal TVKU.
2. Modul Jadwal Siaran, memuat informasi jadwal program TV secara dinamis.
3. Modul Live Streaming, menyediakan layanan siaran langsung berbasis embed player dengan optimasi lazy loading.
4. Modul Dashboard Admin, digunakan oleh staf internal untuk melakukan manajemen konten, termasuk unggah berita dan jadwal siaran.

Setiap modul dikembangkan menggunakan pendekatan modular untuk memastikan efisiensi dan konsistensi tampilan antar halaman. Struktur

umum arsitektur sistem diilustrasikan pada Gambar 3.2, sedangkan alur interaksi antar modul ditampilkan pada Gambar 3.3.



Gambar 16. *Struktur Arsitektur Umum Sistem Frontend TVKU yang Diusulkan*



Gambar 17. *Alur Interaksi Antar Modul dalam Sistem Frontend TVKU*

3.2.2 Tujuan Sistem yang Diusulkan

Tujuan utama dari sistem yang diusulkan adalah:

1. Meningkatkan efisiensi kinerja melalui refaktorisasi struktur kode dan optimalisasi build process.
2. Meningkatkan konsistensi antarmuka dengan menerapkan sistem desain berbasis utility-first CSS framework.
3. Mempercepat proses pemeliharaan dengan membagi tanggung jawab antar modul secara terpisah.

4. Mengurangi technical debt melalui penerapan standar pengembangan modern dan dokumentasi komponen.
5. Meningkatkan pengalaman pengguna (UX) dengan waktu muat halaman yang lebih cepat dan navigasi yang konsisten.

Menurut Nyirongo et al. (2024), sistem modular yang dikembangkan secara bertahap mampu mengurangi waktu pengembangan hingga 40% dibanding sistem monolitik konvensional. Dengan demikian, rancangan sistem ini tidak hanya memperbaiki performa teknis, tetapi juga meningkatkan skalabilitas dan keberlanjutan sistem dalam jangka panjang.

3.2.3 Arsitektur dan Teknologi yang Digunakan

Sistem frontend TVKU yang diusulkan dibangun dengan arsitektur Next.js 14 (App Router) yang mendukung kombinasi server-side rendering (SSR) dan static site generation (SSG). Integrasi data dilakukan menggunakan React Query untuk pengambilan data eksternal serta Zustand untuk manajemen state lokal.

Selain itu, framework Tailwind CSS digunakan untuk menjaga konsistensi desain, sedangkan proses pembangunan sistem diotomatisasi melalui build tool Vercel sebagai platform deployment.

Tabel 10 berikut menampilkan ringkasan teknologi yang digunakan dalam pengembangan sistem ini.

Tabel 10. *Teknologi yang Digunakan dalam Sistem yang Diusulkan*

Komponen	Teknologi	Fungsi Utama
Framework	Next.js 14	Pengelolaan routing dan rendering SSR/SSG
Library State	Zustand & React Query	Manajemen data lokal dan eksternal
Styling	Tailwind CSS	Utility-first styling untuk UI responsif

Server & Deployment	Server Dinustek (hosting internal TVKU)	Menyediakan lingkungan hosting, menjalankan build Next.js, dan menangani proses deployment pada infrastruktur internal
API Data	REST API Internal TVKU	Penyedia data berita, jadwal, dan metadata
Tool Pengujian	Lighthouse & PageSpeed	Evaluasi performa sistem frontend

Dengan kombinasi arsitektur dan teknologi tersebut, sistem yang diusulkan diharapkan dapat memberikan peningkatan signifikan terhadap performa, efisiensi pengembangan, serta pengalaman pengguna dibandingkan dengan sistem sebelumnya. Desain ini juga menjadi dasar bagi tahap selanjutnya, yaitu penerapan metode yang diusulkan menggunakan *Prototype Development Life Cycle* untuk membangun dan menguji sistem secara bertahap.

3.3 Metode yang Diusulkan

Penelitian ini menggunakan pendekatan Extreme Programming (XP) sebagai metode pengembangan perangkat lunak. XP merupakan model pengembangan iteratif dan incremental yang berfokus pada kecepatan adaptasi, kolaborasi intens, serta kualitas perangkat lunak melalui praktik rekayasa yang terstruktur.

Model ini dipilih karena sesuai dengan karakteristik proyek refaktorisasi frontend TVKU, yang menuntut proses perbaikan berulang, respons cepat terhadap perubahan kebutuhan, serta peningkatan performa antarmuka pengguna.

Menurut Beck & Andres (2024), XP efektif diterapkan pada proyek yang membutuhkan fleksibilitas tinggi serta peningkatan kualitas kode secara berkelanjutan. Dalam konteks penelitian ini, XP mendukung perbaikan bertahap terhadap arsitektur frontend, penyederhanaan komponen, serta

penguatan performa tanpa mengganggu operasional sistem yang sudah berjalan.

3.3.1 Tahap Planning

Tahap planning bertujuan untuk menetapkan kebutuhan dan ruang lingkup perbaikan. Aktivitas yang dilakukan meliputi:

1. Pengumpulan kebutuhan pengguna melalui observasi langsung serta diskusi dengan tim redaksi TVKU.
2. Penyusunan user stories untuk menggambarkan kebutuhan dari sudut pandang pengguna, misalnya:
 - a. “Sebagai admin, saya ingin mengunggah berita dengan cepat dan tanpa error.”
 - b. “Sebagai pengguna, saya ingin halaman memuat dengan cepat dan stabil.”
3. Penentuan prioritas menggunakan pendekatan MoSCoW, sehingga setiap kebutuhan memiliki tingkat kepentingan yang jelas.

3.3.2 Tahap Design

Tahap design dilakukan dengan prinsip simple design, yaitu membangun solusi yang paling sederhana namun tetap memenuhi kebutuhan. Prinsip ini menghindarkan sistem dari kerumitan yang tidak perlu serta mengurangi potensi technical debt.

Aktivitas yang dilakukan pada tahap ini meliputi:

1. Perancangan ulang arsitektur frontend berbasis Next.js dengan pendekatan komponen modular.
2. Penyusunan design system menggunakan Tailwind CSS untuk memastikan konsistensi tampilan.
3. Perancangan struktur state management menggunakan React Query dan Zustand agar alur data lebih terorganisir.
4. Pembuatan sketsa komponen (component-driven design) untuk memudahkan proses implementasi dan pemeliharaan.

Pendekatan desain sederhana terbukti meningkatkan kecepatan pengembangan serta mempermudah proses refaktorisasi (Sharma, 2024).

3.3.3 Tahap Coding

Tahap coding dilakukan dengan menerapkan praktik rekayasa perangkat lunak yang menjadi ciri khas XP, yaitu:

1. Pair Programming (disesuaikan)

Karena penelitian dilakukan dalam skala individu, praktik ini diadaptasi menjadi code review internal dengan pihak TVKU untuk memastikan kualitas kode tetap terjaga.

2. Test-Driven Development (TDD)

Sebelum proses penulisan kode, dibuat terlebih dahulu test case untuk memastikan bahwa setiap fungsi dan komponen berjalan sesuai kebutuhan.

3. Continuous Refactoring

Refaktorisasi dilakukan secara terus-menerus untuk merapikan struktur kode, menghilangkan duplikasi, meningkatkan efisiensi, dan memperbaiki performa.

4. Continuous Integration

Setiap perubahan kode diuji secara otomatis melalui pipeline Vercel untuk memastikan stabilitas aplikasi pada setiap iterasi.

3.3.4 Tahap Testing

Tahap testing dilakukan secara berkelanjutan sepanjang proses pengembangan (continuous testing). Pengujian yang dilakukan meliputi:

1. Unit Testing

Menguji setiap komponen atau fungsi secara terpisah untuk memastikan logika berjalan dengan benar.

2. Integration Testing

Menguji integrasi antara frontend dan API untuk memastikan alur data berjalan sesuai harapan.

3. Functional/Black-Box Testing

Menguji fungsi sistem berdasarkan skenario pada user stories tanpa melihat struktur internal kode.

4. Performance Testing

Pengukuran performa dilakukan menggunakan Lighthouse dan Core Web Vitals meliputi:

- a. Largest Contentful Paint (LCP)
- b. Cumulative Layout Shift (CLS)
- c. First Input Delay (FID)
- d. Kecepatan loading halaman

5. User Acceptance Testing (UAT)

Melibatkan tim redaksi TVKU untuk menilai kelayakan, kenyamanan, dan kemudahan penggunaan antarmuka.

Model continuous testing dalam XP terbukti mampu meningkatkan kualitas sistem secara konsisten (Nyirongo et al., 2024).

3.3.5 Tahap Release

Tahap release merupakan tahap akhir dari setiap iterasi. Aktivitas pada tahap ini mencakup:

1. Deployment aplikasi ke platform Vercel sebagai lingkungan produksi.
2. Penyusunan dokumentasi komponen dan panduan penggunaan bagi pengguna internal.
3. Monitoring performa setelah rilis menggunakan metrik Core Web Vitals.
4. Pengumpulan feedback pengguna, yang selanjutnya digunakan sebagai dasar iterasi berikutnya.

3.4 Metode Pengujian

3.4.1 Pengujian Unit (Unit Testing)

Pengujian unit dilakukan untuk memastikan bahwa setiap komponen atau fungsi kecil pada sistem berjalan sesuai dengan logika yang diharapkan. Pengujian ini berfokus pada komponen UI, fungsi

utilitas, serta bagian-bagian kecil dari alur data. Unit testing membantu mendeteksi kesalahan pada level paling awal dan mempermudah proses refaktorisasi.

3.4.2 Pengujian Integrasi (Integration Testing)

Pengujian integrasi dilakukan untuk memastikan bahwa interaksi antara frontend dan API TVKU berjalan secara konsisten dan stabil.

Elemen yang diuji meliputi:

1. Komunikasi data melalui REST API
2. Respons server terhadap permintaan frontend
3. Penanganan error dan fallback UI

Pengujian ini penting untuk memvalidasi kelancaran alur data setelah proses refaktorisasi dilakukan.

3.4.3 Pengujian Fungsional (Functional / Black-Box Testing)

Pengujian fungsional dilakukan tanpa melihat struktur internal kode, melainkan berdasarkan skenario penggunaan (user stories).

Pengujian ini memastikan bahwa fitur yang dibangun benar-benar memenuhi kebutuhan pengguna, seperti:

1. proses unggah berita
2. proses edit dan manajemen konten
3. navigasi antarmuka
4. stabilitas layout

Metode black-box testing digunakan karena mudah diterapkan dan efektif untuk memvalidasi perilaku sistem dari sudut pandang pengguna.

3.4.4 Pengujian Performa (Performance Testing)

Pengujian performa dilakukan untuk mengevaluasi kecepatan, stabilitas, dan efisiensi frontend setelah proses refaktorisasi.

Alat yang digunakan adalah Lighthouse dan Core Web Vitals, dengan fokus pada indikator berikut:

1. Largest Contentful Paint (LCP) → kecepatan elemen utama muncul
2. Cumulative Layout Shift (CLS) → stabilitas tata letak

3. First Input Delay (FID) / Interaction to Next Paint (INP) →
responsivitas
4. Total Blocking Time (TBT)
5. Kecepatan loading keseluruhan

3.4.5 User Acceptance Testing (UAT)

UAT dilakukan dengan melibatkan tim redaksi TVKU sebagai pengguna utama. Pengujian ini mengevaluasi:

1. Kemudahan penggunaan antarmuka
2. Kejelasan navigasi
3. Kecepatan akses
4. Kesesuaian sistem dengan kebutuhan operasional harian

BAB IV

HASIL PENELITIAN DAN PEMBAHASAN

4.1 Analisis Sistem dan Kebutuhan

Bab ini menyajikan hasil penelitian serta pembahasan yang diperoleh dari seluruh rangkaian proses analisis, perancangan, implementasi, dan pengujian sistem frontend TVKU. Pembahasan disusun secara sistematis dan terstruktur untuk menunjukkan hubungan yang jelas antara permasalahan awal yang ditemukan pada sistem sebelumnya dengan solusi yang diusulkan melalui proses refaktorisasi frontend. Selain itu, bab ini juga menekankan pada bagaimana penerapan teknologi modern dan pendekatan pengembangan yang digunakan mampu meningkatkan kualitas sistem dari sisi performa, maintainability, serta pengalaman pengguna.

Bab IV menjadi bagian utama yang memperlihatkan kontribusi nyata dari penelitian ini, karena pada bab inilah hasil implementasi dan pengujian sistem dijabarkan secara rinci. Seluruh hasil yang disajikan pada bab ini dikaitkan kembali dengan tujuan penelitian dan metodologi yang telah dijelaskan pada bab sebelumnya, sehingga alur penelitian dapat dipahami secara utuh dan logis.

4.1.1 Gambaran Umum Objek Penelitian

TVKU merupakan stasiun televisi komunitas milik Universitas Dian Nuswantoro (UDINUS) yang berfungsi sebagai media informasi dan sarana pembelajaran mahasiswa. Dalam operasional digitalnya, TVKU menyediakan layanan portal berita, jadwal program, dan live streaming yang diakses oleh audiens melalui platform berbasis web.

4.1.2 Permasalahan Teknis Sistem Lama

Berdasarkan hasil observasi, sistem frontend TVKU saat ini masih mengandalkan arsitektur campuran yang tidak konsisten antara framework modern (React/Next.js) dengan elemen warisan (legacy) seperti jQuery dan CSS monolitik. Kondisi ini menimbulkan sejumlah

permasalahan teknis yang berdampak pada performa, maintainability, dan pengalaman pengguna.

4.1.2.1 Identifikasi Masalah pada Arsitektur Lama

Tabel 11. *Permasalahan teknis legacy*

Aspek	Kondisi Sistem Lama	Dampak Teknis
Manipulasi DOM	Pencampuran paradigma deklaratif (React) dengan imperatif (jQuery).	Terjadi inkonsistensi tampilan dan kesalahan logika interaksi pengguna.
Manajemen State	Data dikelola secara terpisah di tiap komponen (<i>local state</i>) tanpa mekanisme <i>caching</i> .	Muncul fenomena <i>prop drilling</i> yang memperumit pelacakan alur data.
Gaya Visual	Penggunaan berkas CSS monolitik dari tema lama.	Tingginya konflik aturan gaya dan sulitnya kustomisasi antarmuka.
Performa Web	Pemuatan berkas pustaka eksternal yang besar dan tidak terpakai (<i>unused library</i>).	Metrik LCP 65% lebih lambat dari standar ideal, menurunkan kepuasan pengguna.

4.1.2.2 Analisis Kebutuhan untuk Refaktorisasi

Untuk mengatasi permasalahan teknis yang teridentifikasi, diperlukan refaktorisasi sistem dengan batasan fungsional dan standar kualitas (non-fungsional) sebagai berikut:

a. Analisis Kebutuhan Fungsional

1. Akses Konten Berita: Pengguna dapat membaca daftar dan rincian berita terbaru secara dinamis yang bersumber dari API TVKU.
2. Informasi Jadwal Siaran: Pengguna dapat melihat jadwal program acara televisi secara terstruktur dan aktual.

3. Layanan Live Streaming: Pengguna dapat menikmati siaran televisi secara langsung melalui pemutar video yang terintegrasi di web.

b. Analisis Kebutuhan Non-Fungsional

1. Optimalisasi Performa: Mencapai standar metrik Core Web Vitals yang ideal ($LCP \leq 2,5$ detik, $FID < 100$ ms, dan $CLS \leq 0,1$)
2. Maintainability: Struktur kode harus modular berbasis komponen agar mudah dipelihara dan dikembangkan di masa mendatang.
3. Efisiensi Resource: Pengurangan ukuran berkas aplikasi (bundle size) melalui penghapusan dependensi jQuery dan optimasi aset.
4. Konsistensi UI: Antarmuka harus seragam dan responsif di berbagai perangkat dengan menerapkan Tailwind CSS.

Tabel 12. *Kebutuhan Fungsional dan non fungsional*

No	Jenis Kebutuhan	Aspek / Parameter	Deskripsi & Standar Pencapaian
1	Fungsional	Akses Konten Berita	Pengguna dapat membaca daftar dan rincian berita terbaru secara dinamis dari API TVKU.
2	Fungsional	Informasi Jadwal Siaran	Pengguna dapat melihat jadwal program acara televisi secara terstruktur dan aktual.
3	Fungsional	Layanan Live Streaming	Pengguna dapat menikmati siaran langsung melalui pemutar video terintegrasi di web.
4	Non-Fungsional	Optimalisasi Performa	Mencapai standar Core Web Vitals: $LCP \leq 2,5$ detik, $FID < 100$ ms, dan $CLS \leq 0,1$.
5	Non-Fungsional	<i>Maintainability</i>	Struktur kode modular berbasis komponen untuk kemudahan pemeliharaan jangka panjang.
6	Non-Fungsional	Efisiensi <i>Resource</i>	Reduksi <i>bundle size</i> dengan menghapus jQuery dan melakukan optimasi aset (gambar/skrip).
7	Non-Fungsional	Konsistensi UI	Antarmuka seragam, responsif, dan modern dengan implementasi Tailwind CSS.

4.2 Perancangan Sistem Modern

4.2.1 Perancangan Arsitektur Next.js 14

Sistem dirancang menggunakan framework Next.js 14 dengan pendekatan App Router untuk mendukung optimasi Server-Side Rendering (SSR) dan Static Site Generation (SSG). Arsitektur ini dibagi menjadi tiga lapisan utama untuk memisahkan tanggung jawab kode:

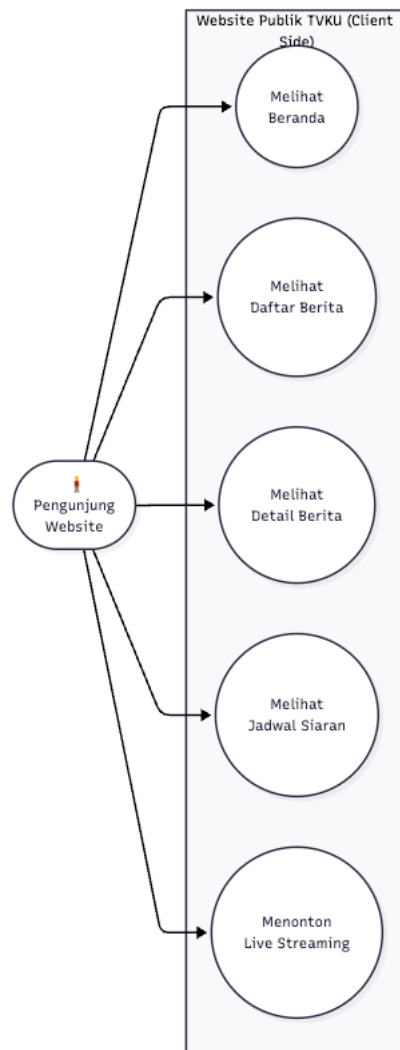
1. **Presentation Layer:** Berisi komponen antarmuka yang berinteraksi langsung dengan pengguna.
2. **Logic/State Layer:** Mengatur pengambilan data (fetching) dan manajemen kondisi aplikasi.
3. **Data Layer:** Terhubung dengan API eksternal TVKU untuk menyediakan data dinamis.

4.2.2 Pemodelan UML (Sistem Baru)

Pemodelan Unified Modeling Language (UML) digunakan untuk memvisualisasikan perilaku dan interaksi komponen sistem.

4.2.2.1 Use Case Diagram

Use Case Diagram menggambarkan interaksi pengguna publik dengan fungsi-fungsi utama sistem TVKU, meliputi akses berita, melihat jadwal, dan menonton live streaming.



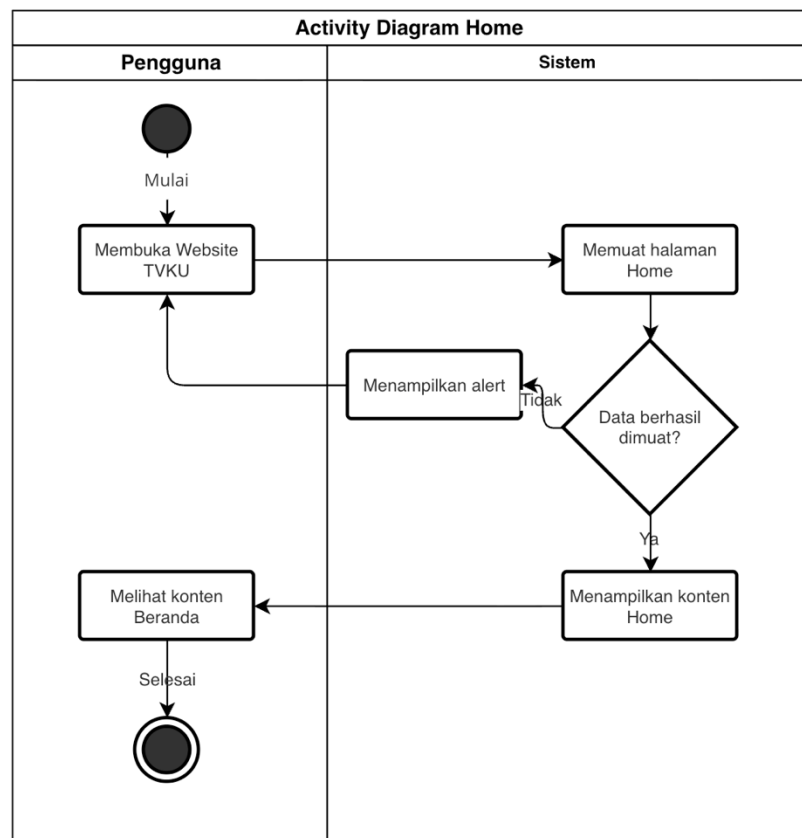
Gambar 18. *Use Case Diagram Sistem TVKU*

4.2.2.2 Activity Diagram

Activity Diagram menjelaskan alur kerja sistem saat pengguna mengakses konten, mulai dari permintaan data hingga proses rendering tampilan secara otomatis.

1. Activity Diagram Home

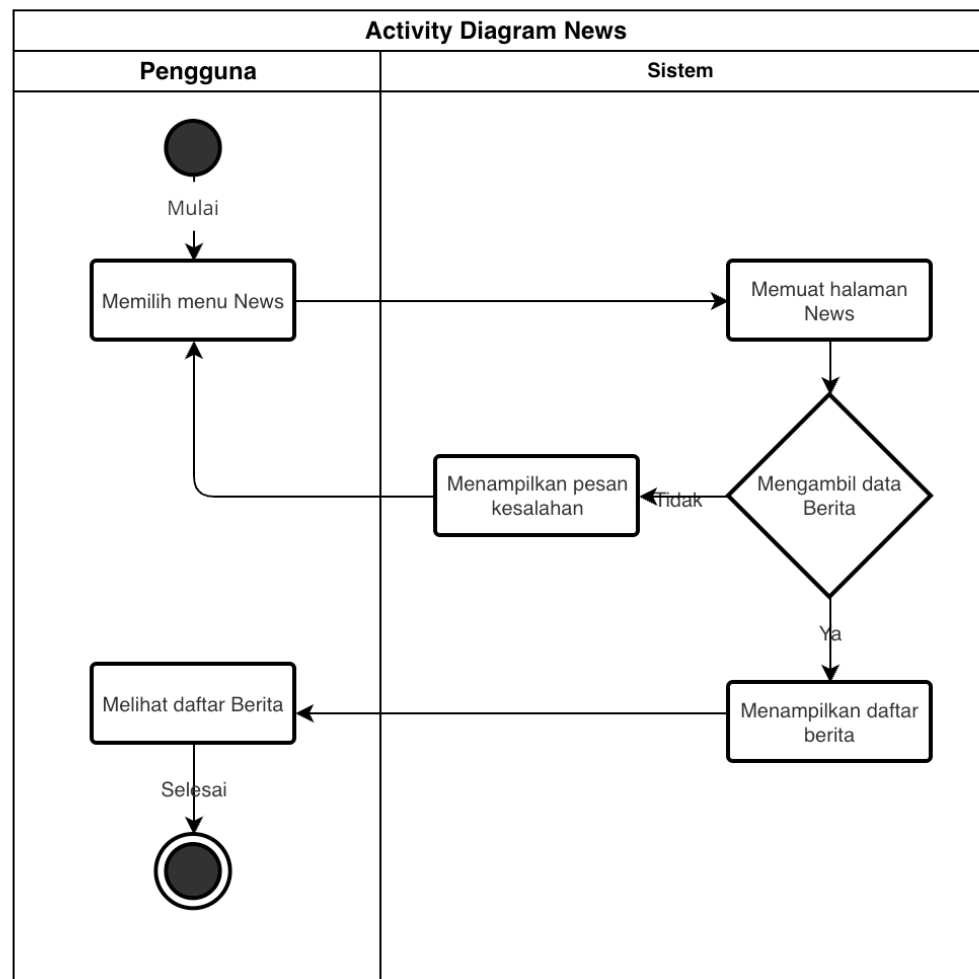
Activity Diagram Home menggambarkan alur interaksi pengguna saat mengakses halaman beranda pada website TVKU. Proses dimulai ketika pengguna membuka website, kemudian sistem memuat halaman Home dan melakukan proses pemuatan data konten utama. Selanjutnya, sistem melakukan pengecekan terhadap keberhasilan pemuatan data. Apabila data berhasil dimuat, sistem menampilkan konten Home kepada pengguna. Namun, apabila data gagal dimuat, sistem menampilkan pesan kesalahan kepada pengguna. Proses berakhir ketika pengguna berhasil melihat konten beranda yang ditampilkan oleh sistem.



Gambar 19. Activity Diagram Home

2. Activity Diagram News

Activity Diagram News menggambarkan alur pengguna dalam mengakses halaman berita pada website TVKU. Proses dimulai saat pengguna memilih menu News, kemudian sistem memuat halaman, mengambil data berita, dan menampilkan hasil atau pesan kesalahan sebelum proses berakhir.

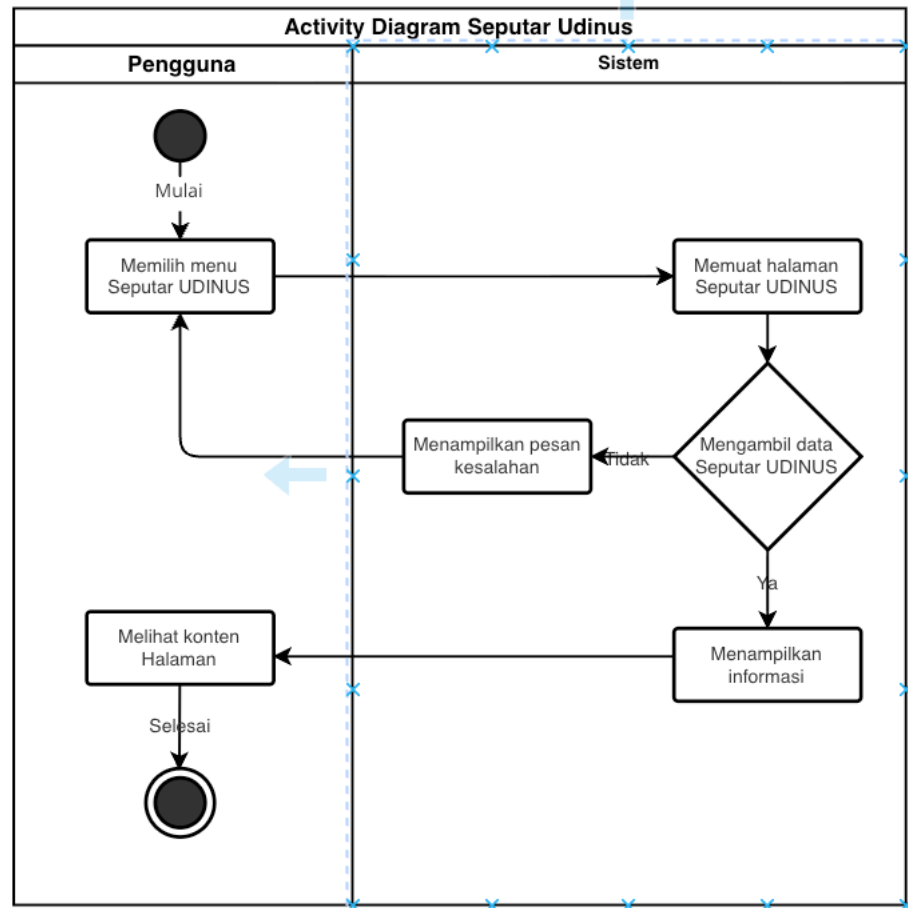


Gambar 20. Activity Diagram News

3. Activity Diagram Seputar Udinus

Activity Diagram Seputar Udinus menggambarkan alur pengguna dalam mengakses informasi Seputar Udinus pada website TVKU. Proses dimulai ketika pengguna memilih menu

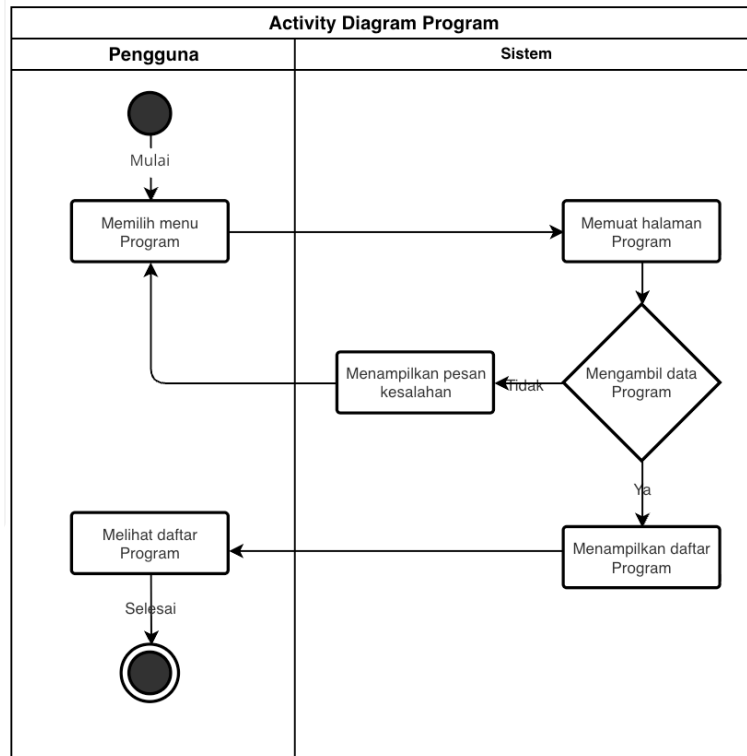
Seputar Udinus, kemudian sistem memuat halaman dan mengambil data informasi untuk ditampilkan atau menampilkan pesan kesalahan sebelum proses berakhir.



Gambar 21. Activity Diagram Seputar Udinus

4. Activity Diagram Program

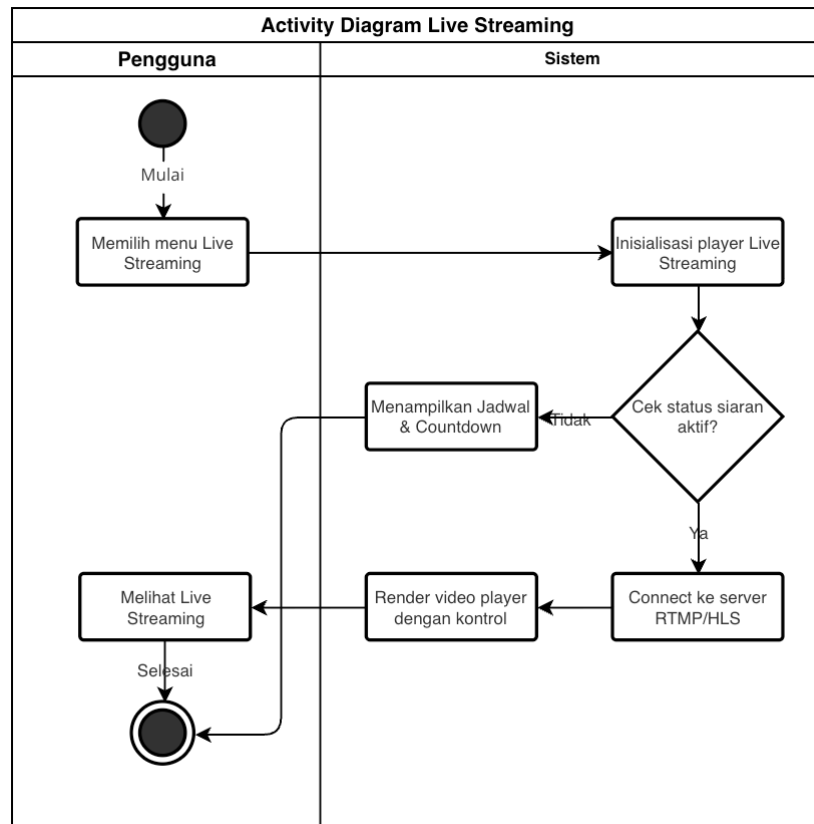
Activity Diagram Program menggambarkan alur pengguna dalam mengakses halaman Program pada website TVKU. Proses dimulai ketika pengguna memilih menu Program, kemudian sistem memuat halaman dan mengambil data program untuk ditampilkan atau menampilkan pesan kesalahan sebelum proses berakhir.



Gambar 22. *Activity Diagram Program*

5. Activity Diagram Live Streaming

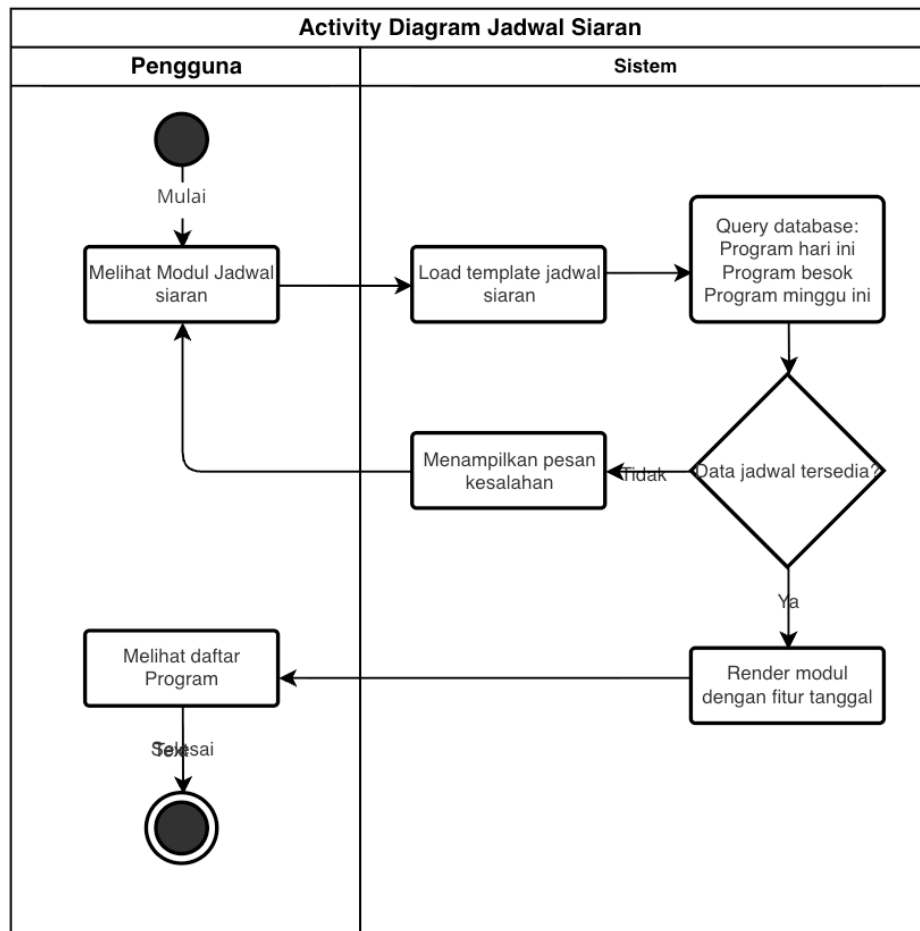
Activity Diagram Live Streaming menggambarkan alur pengguna dalam mengakses fitur siaran langsung pada website TVKU. Proses dimulai ketika pengguna memilih menu Live Streaming, kemudian sistem memuat halaman dan melakukan pengecekan status siaran untuk menampilkan video siaran langsung atau informasi bahwa siaran tidak tersedia sebelum proses berakhir.



Gambar 23. *Activity Diagram Live Streaming*

6. Activity Diagram Jadwal Siaran

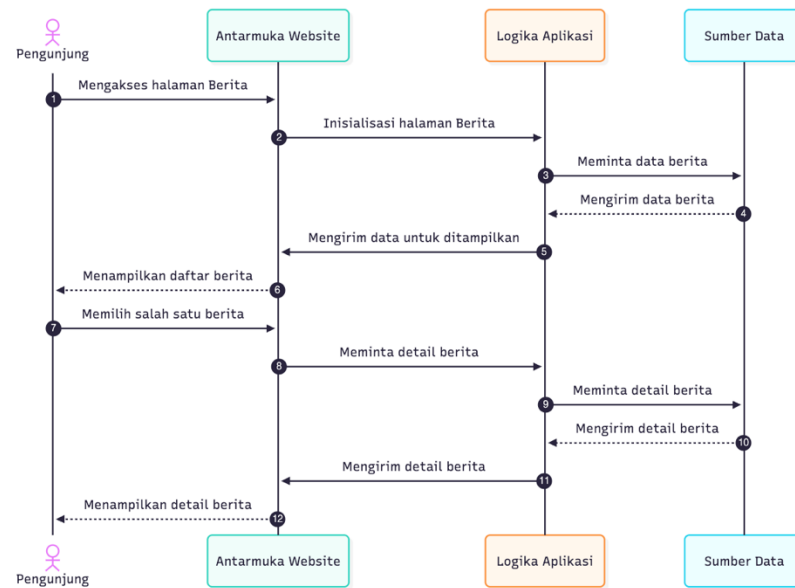
Activity Diagram Jadwal Siaran menggambarkan alur pengguna dalam mengakses informasi jadwal acara pada website TVKU. Proses dimulai ketika pengguna memilih menu Jadwal Siaran, kemudian sistem memuat halaman dan mengambil data jadwal untuk ditampilkan atau menampilkan pesan kesalahan sebelum proses berakhir.



Gambar 24. *Activity Diagram Jadwal Siaran*

4.2.2.3 Sequence Diagram (Alur Data API)

Sequence Diagram memvisualisasikan urutan komunikasi antara komponen frontend dengan API backend TVKU saat melakukan *pengambilan data berita dan jadwal*.

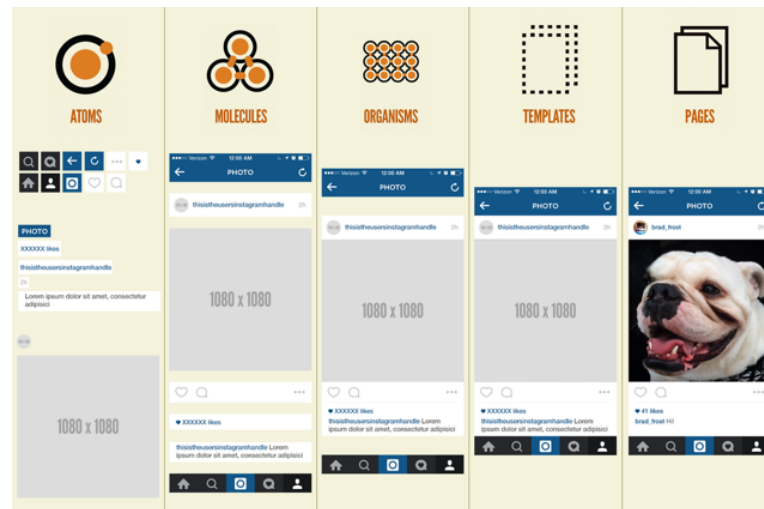


Gambar 25. Use Case Diagram Sistem TVKU

4.2.3 Perancangan Antarmuka Berbasis Atomic Design

Struktur antarmuka dirancang menggunakan prinsip Atomic Design untuk meningkatkan reusability komponen. Komponen dibagi menjadi unit terkecil (Atoms) hingga unit kompleks (Organisms dan Templates).

1. Atoms: Elemen dasar seperti tombol, input, dan label.
2. Molecules: Gabungan beberapa atoms, misalnya kolom pencarian.
3. Organisms: Komponen kompleks seperti header atau kartu berita (NewsCard).



Gambar 26. Hierarki Atomic Design pada UI TVKU

4.2.4 Perancangan Manajemen State Terpisah

4.2.4.1 Server State Management (React Query)

Server state mencakup seluruh data yang berasal dari API eksternal TVKU, seperti daftar berita dan jadwal siaran. Perancangan menggunakan React Query difokuskan pada sinkronisasi data secara deklaratif dengan fitur utama sebagai berikut:

1. Mekanisme Caching: Menyimpan data di memori lokal untuk mengurangi jumlah permintaan jaringan yang berulang.
2. Background Synchronization: Melakukan pembaruan data di latar belakang agar informasi yang ditampilkan tetap aktual tanpa mengganggu interaksi pengguna.
3. Pengelolaan Status: Menangani kondisi loading, error, dan success secara otomatis melalui hook khusus untuk setiap modul.

4.2.4.2 Client State Management (Zustand)

Client state digunakan untuk mengelola kondisi internal aplikasi yang bersifat sinkron dan hanya relevan di sisi pengguna. Zustand dipilih karena memiliki sintaks yang lebih sederhana dan beban memori yang lebih ringan dibandingkan Redux. Perancangan ini mencakup:

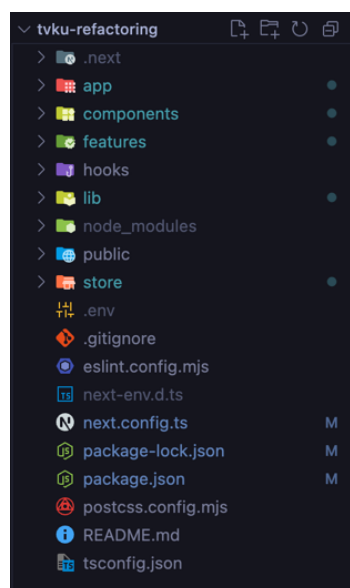
1. Centralized Store: Menyimpan status antarmuka seperti status aktif navigasi, status pemutar video, dan tampilan modal dalam satu store terpusat.
2. Hook-based Access: Memungkinkan setiap komponen untuk berlangganan pada bagian state tertentu secara langsung, sehingga menghindari fenomena prop drilling.
3. Efisiensi Rendering: Komponen hanya akan melakukan render ulang jika bagian state yang mereka gunakan mengalami perubahan.

4.3 Tahapan Refaktorisasi Sistem

Tahap implementasi merupakan realisasi dari perancangan arsitektur modern ke dalam baris kode program. Fokus utama pada tahap ini adalah melakukan migrasi dari kode *legacy* yang imperatif menuju struktur komponen yang deklaratif dan modular.

4.3.1 Restrukturisasi Proyek

Struktur proyek disusun menggunakan standar Next.js 14 App Router untuk memastikan pemisahan tanggung jawab yang jelas antar modul. Direktori utama dibagi menjadi /app untuk routing, /components untuk unit antarmuka, dan /hooks untuk logika pengambilan data.



Gambar 27. Hierarki Folder Sesudah Refaktorisasi

Transformasi ini menghilangkan ketergantungan antar berkas yang tidak teratur, sehingga mempermudah proses pemeliharaan dan pelacakan bug.

4.3.2 Modularisasi Komponen (Before vs After):

Refaktorisasi dilakukan dengan mengganti manipulasi DOM secara langsung menggunakan jQuery menjadi komponen React yang deklaratif. Hal ini bertujuan untuk menghilangkan technical debt dan beban pustaka yang tidak efisien.

Tabel 13. *Perbandingan Pola Kode Legacy vs Modern*

Aspek	Kode Legacy (jQuery)	Kode Modern (Next.js/React)
Manipulasi UI	Menggunakan selektor seperti <code>\$('.class').html()</code> untuk mengubah konten.	Menggunakan <i>state</i> dan <i>props</i> yang merender UI secara otomatis.
Data Fetching	Menggunakan <code>\$.ajax()</code> di dalam skrip global yang sulit di-debug.	Menggunakan <i>custom hooks</i> berbasis React Query yang mendukung <i>caching</i> .
Styling	Bergantung pada berkas CSS eksternal berukuran besar.	Menggunakan Tailwind CSS yang hanya menyertakan kelas yang digunakan (<i>purge</i>).

4.3.2.1 Implementasi kode modern

a. Penggunaan State dan Props untuk Rendering UI Otomatis

Sistem dibangun menggunakan React.js yang memanfaatkan konsep State dan Props untuk merender komponen secara dinamis. State digunakan untuk mengelola data internal komponen yang dapat berubah (seperti status loading atau data navigasi), sedangkan Props digunakan untuk mengirimkan data antar komponen. Berikut adalah implementasi

pada komponen `src/components/navbar.tsx` yang menggunakan `useState` untuk mengatur status menu responsif (mobile) dan event scroll.

```
const Navbar = () => {
  // State untuk mendeteksi scroll dan status menu mobile
  const [scroll, setScroll] = useState(false);
  const [isMenuOpen, setIsMenuOpen] = useState(false);

  // Effect untuk mendeteksi pergerakan scroll window
  useEffect(() => {
    const handleScroll = () => {
      setScroll(window.scrollY > 50);
    };
    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll);
  }, []);

  // Fungsi toggle statis menu
  const toggleMenu = () => setIsMenuOpen(!isMenuOpen);

  return (
    // Render UI berdasarkan state isMenuOpen
    <nav className={isMenuOpen ? "block" : "hidden"}>
      { /* ... item navigasi ... */ }
    </nav>
  );
};
```

Gambar 28. Implementasi State dan Props

b. Implementasi Custom Hooks Berbasis React Query dengan Dukungan Caching

Untuk menangani pengambilan data (data fetching) dari API, sistem menggunakan library `@tanstack/react-query`. Implementasi ini dibungkus dalam sebuah Custom Hook untuk memisahkan logika bisnis dari tampilan (UI). Pendekatan ini memberikan keuntungan berupa caching otomatis, dimana data yang sudah diambil akan disimpan sementara, sehingga navigasi antar halaman menjadi lebih cepat tanpa perlu memuat ulang data yang sama. Contoh implementasinya terdapat pada `src/hooks/usePrograms.ts` dan digunakan pada `src/components/program.tsx`.

```
import { useQuery } from "@tanstack/react-query";
import { fetchWithFallback } from "@lib/fetchWithFallback";

export const usePrograms = () => {
  return useQuery({
    queryKey: ["programs"], // Key unik untuk caching
    queryFn: async () => {
      // Logika fetch data dengan fallback mekanism
      const { data, error } = await fetchWithFallback(
        `${process.env.NEXT_PUBLIC_API_BASE_URL}/our-programs`,
        "/data/our-programs.json"
      );
      if (error) throw new Error(error);
      return data?.data || [];
    },
    staleTime: 60000, // Data dianggap fresh selama 1 menit
  });
};
```

Gambar 30. Implementasi Custom Hooks

```
export const Program = () => {
  // Menggunakan custom hook untuk mendapatkan data dan status loading
  const { data: programs, isLoading, error } = usePrograms();

  if (isLoading) return <p>Loading programs...</p>;
  if (error) return <p>Error loading data.</p>;

  // Render list program secara otomatis saat data tersedia
  return (
    <div className="program-list">
      {programs.map((program) => (
        <Card key={program.id} title={program.judul} thumbnail=
{program.thumbnail} />
      ))}
    </div>
  );
};
```

Gambar 29. Penggunaan Hook

c. Penggunaan Tailwind CSS dengan Optimasi Purge

Desain antarmuka diimplementasikan menggunakan Tailwind CSS versi 4. Versi ini menggunakan engine "Oxide" yang secara otomatis mendeteksi kelas-kelas utilitas yang digunakan dalam file kode

layout.tsx dan hanya menyertakan CSS yang benar-benar terpakai ke dalam bundle produksi. Teknik ini (Tree Shaking) memastikan ukuran file CSS tetap kecil meskipun menggunakan framework yang komprehensif.

```
@import "tailwindcss";

/* Definisi tema kustom yang terintegrasi langsung */
@theme inline {
  --color-primary: oklch(0.435 0.166 260.031); /* Warna brand utama */
  --color-secondary: oklch(0.85 0.16 85); /* Warna aksen */
  --radius-lg: 0.5rem;
}

/* Utilitas kustom */
@layer base {
  body {
    @apply bg-background text-foreground antialiased;
  }
}
```

Gambar 31. Penggunaan TailwindCSS

4.3.3 Optimasi Performa Aplikasi

Selain transformasi kode, dilakukan penerapan teknik optimasi untuk memastikan target performa tercapai:

4.3.3.1 Code Splitting

Membagi berkas JavaScript ke dalam unit yang lebih kecil sehingga peramban hanya memuat kode yang dibutuhkan.

4.3.3.2 Tree Shaking

Menghapus fungsi atau pustaka yang tidak digunakan dalam proses pembuatan build akhir.

4.3.3.3 Image Optimization

Menggunakan komponen `<Image />` dari Next.js untuk konversi otomatis ke format WebP guna menekan ukuran aset.

4.3.4 Implementasi Fitur Utama (Dokumentasi Antarmuka)

4.3.4.1 Halaman Beranda dan Navigasi

Pada tampilan sebelum refaktorisasi, halaman Home masih memiliki struktur layout yang kurang konsisten, dengan penempatan elemen yang belum terorganisir secara optimal. Informasi utama seperti highlight berita dan akses menu belum memiliki hirarki visual yang jelas, sehingga pengguna membutuhkan waktu lebih lama untuk memahami konten yang ditampilkan.



Gambar 32. Halaman Beranda dan Navigasi sebelum Refaktorisasi

Setelah dilakukan refaktorisasi, halaman Home dirancang ulang dengan struktur layout yang lebih rapi dan modular. Hirarki informasi dibuat lebih jelas melalui pengelompokan konten utama, penggunaan whitespace yang lebih proporsional, serta konsistensi komponen antarmuka. Perubahan ini memberikan pengalaman pengguna yang

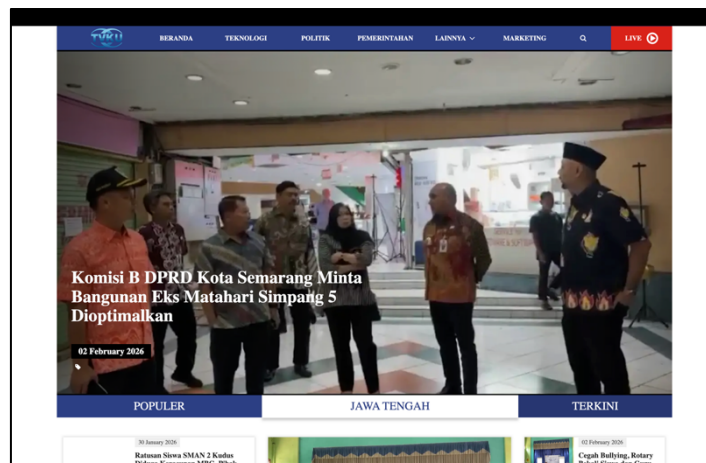
lebih intuitif serta mempermudah pengguna dalam mengakses informasi utama sejak pertama kali membuka website.



Gambar 33. Halaman Beranda dan Navigasi setelah Refactorisasi

4.3.4.2 Halaman Berita

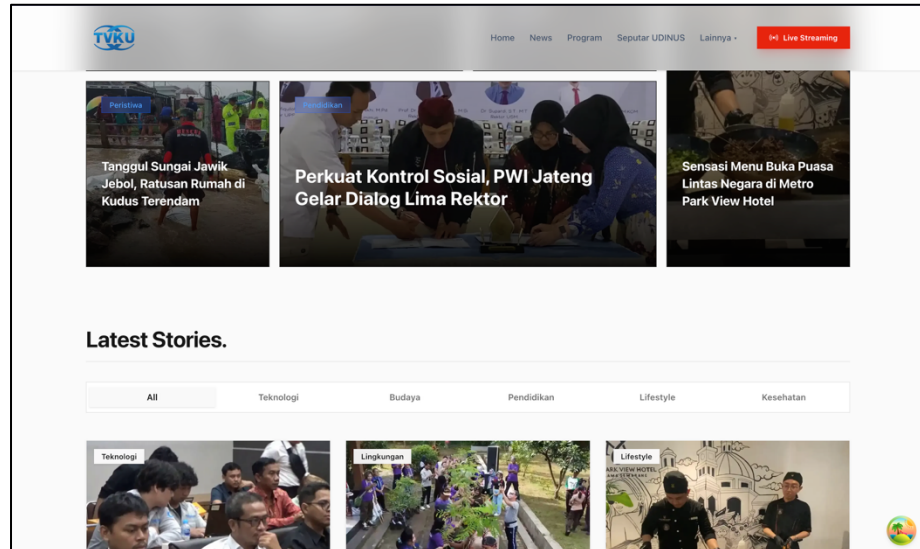
Pada versi sebelum refaktorisasi, halaman News menampilkan daftar berita dengan tampilan yang relatif padat dan kurang responsif terhadap berbagai ukuran layar. Selain itu, komponen kartu berita belum sepenuhnya konsisten dari sisi ukuran, tipografi, dan jarak antar elemen.



Gambar 34. Halaman Berita sebelum Refactorisasi

Pada versi setelah refaktorisasi, halaman News diimplementasikan menggunakan komponen modular yang reusable. Tampilan kartu berita dibuat lebih seragam dengan penyesuaian tipografi dan layout yang responsif. Perubahan ini tidak hanya meningkatkan estetika tampilan,

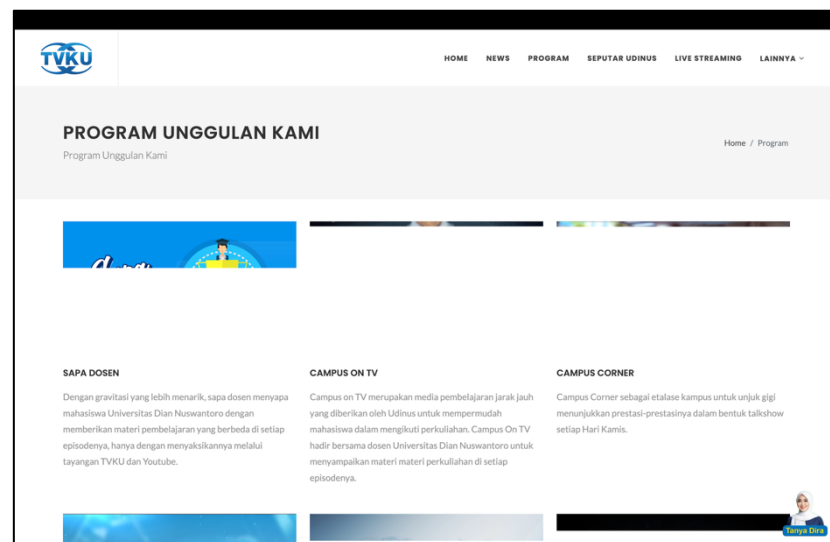
tetapi juga memudahkan pengguna dalam membaca dan menelusuri berita secara lebih nyaman.



Gambar 35. Halaman Berita setelah Refactorisasi

4.3.4.3 Halaman Program

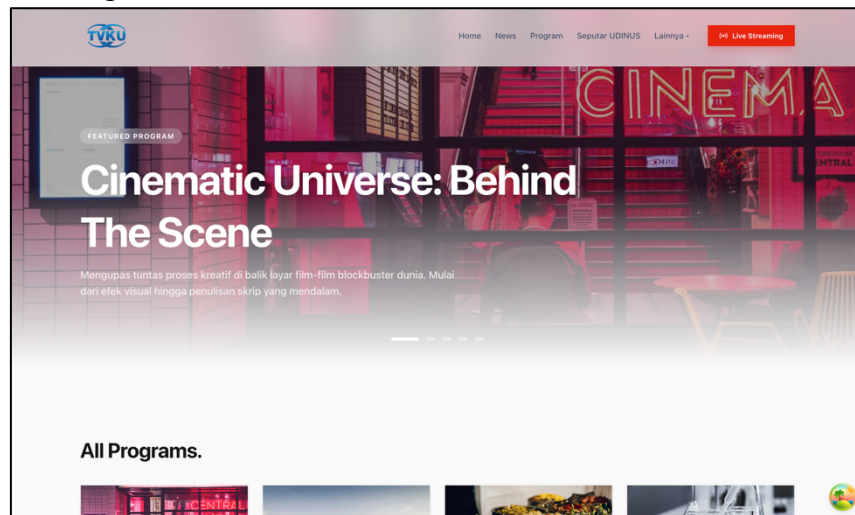
Pada tampilan sebelum refaktorisasi, halaman Program masih menampilkan daftar program acara dengan struktur yang kurang



konsisten dan visual yang belum terstandarisasi. Informasi program seperti judul acara, deskripsi singkat, dan jadwal penayangan belum disajikan dengan hirarki visual yang jelas, sehingga pengguna perlu membaca secara lebih teliti untuk memahami detail setiap program yang tersedia.

Gambar 36. *Halaman Program sebelum Refactorisasi*

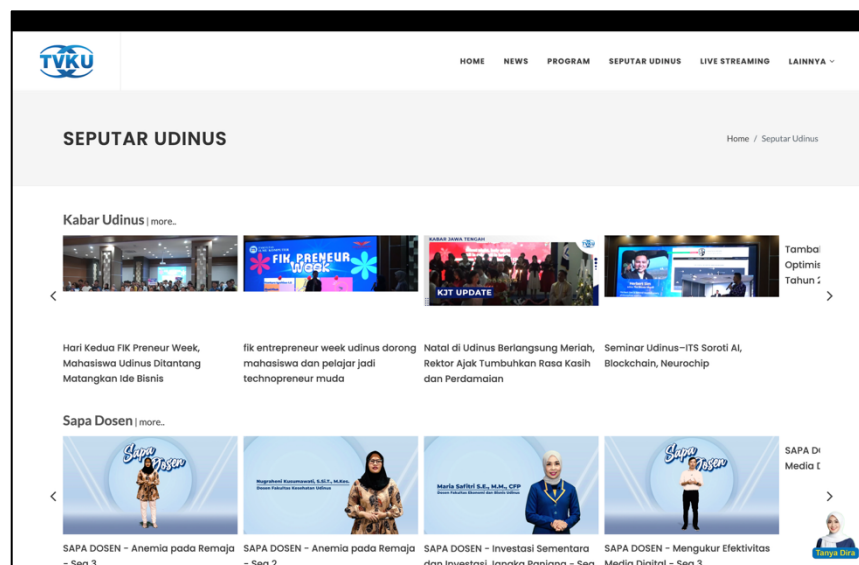
Setelah dilakukan refaktorisasi, halaman Program dirancang ulang menggunakan pendekatan komponen modular dengan tata letak yang lebih terstruktur. Setiap program ditampilkan dalam bentuk kartu (card) dengan pembagian informasi yang jelas antara judul program, deskripsi, dan elemen pendukung lainnya. Konsistensi desain antar kartu program juga diperhatikan untuk menjaga keseragaman tampilan dan meningkatkan keterbacaan informasi.



Gambar 37. *Halaman Program setelah Refactorisasi*

4.3.4.4 Halaman Halaman Seputar Udinus

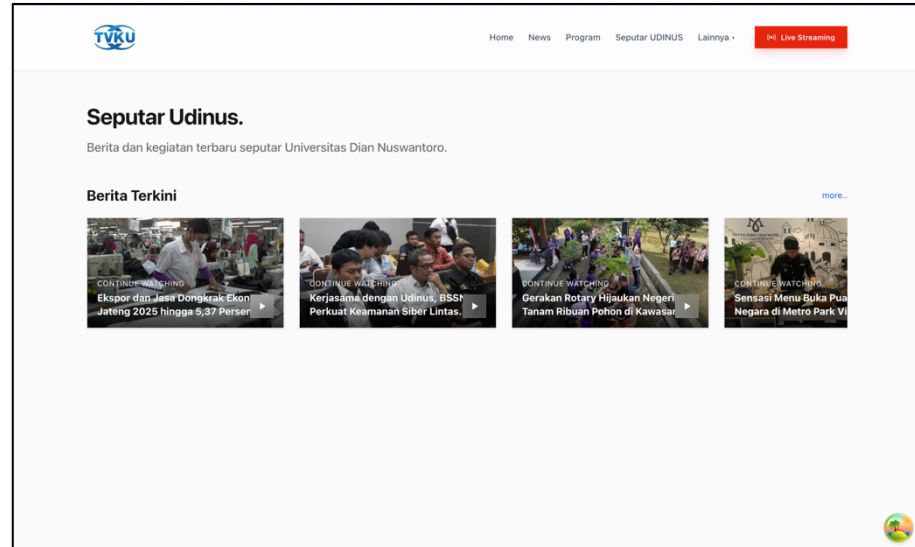
Sebelum refaktorisasi, halaman Seputar Udinus masih memiliki penyajian informasi yang bersifat statis dengan struktur konten yang kurang tersegmentasi. Hal ini menyebabkan informasi yang disajikan kurang mudah dipahami secara cepat oleh pengguna.



Gambar 38. Halaman Seputar UDINUS sebelum Refactorisasi

Setelah refaktorisasi, halaman Seputar Udinus ditampilkan dengan struktur konten yang lebih terorganisir. Informasi dibagi ke dalam beberapa section yang jelas, disertai dengan konsistensi desain dan penggunaan komponen yang seragam dengan halaman lain. Pendekatan

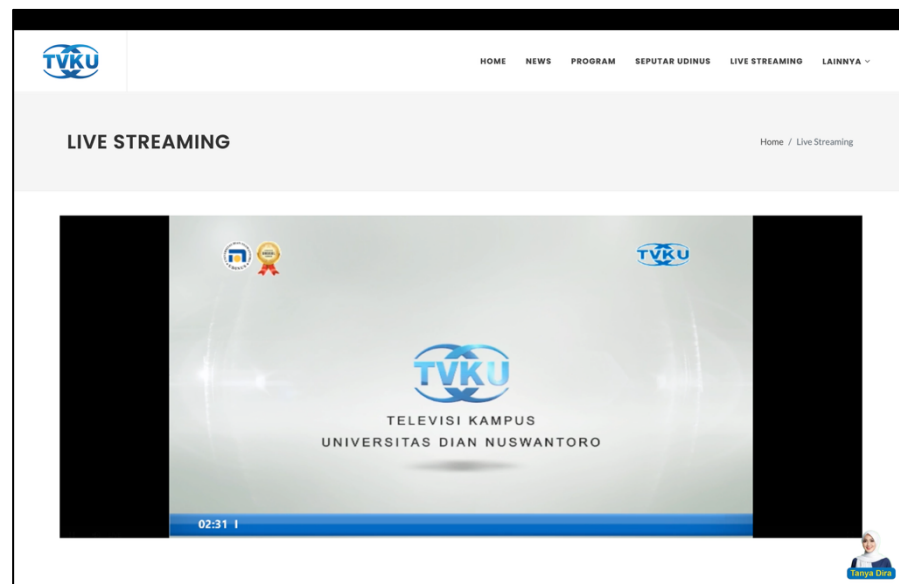
ini meningkatkan keterbacaan konten serta menjaga konsistensi identitas visual sistem frontend TVKU.



Gambar 39. Halaman Seputar UDINUS setelah Refactorisasi

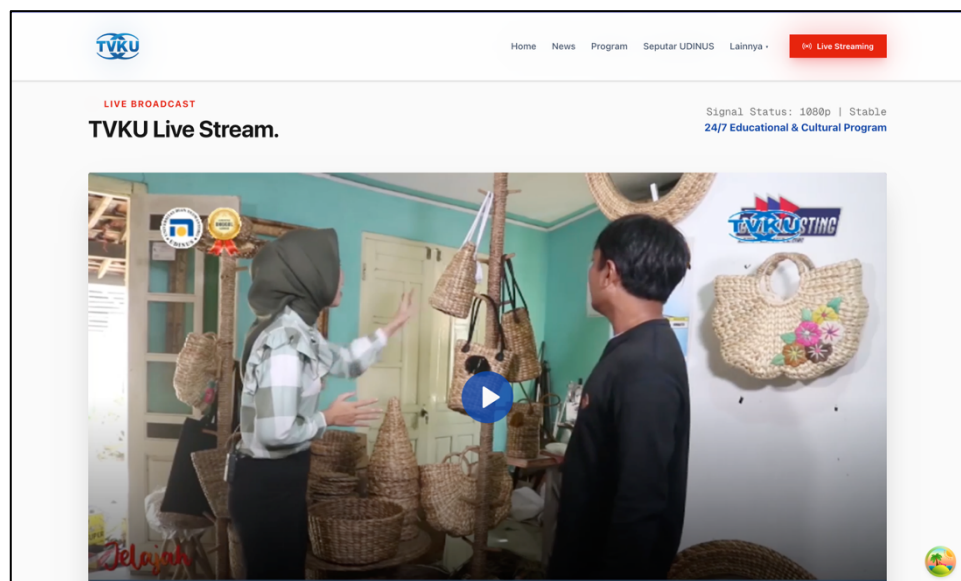
4.3.4.5 Halaman Live Streaming

Pada tampilan sebelum refaktorisasi, halaman Live Streaming belum memiliki fokus visual yang optimal terhadap konten utama, yaitu video siaran langsung. Elemen pendukung seperti navigasi dan informasi tambahan masih berpotensi mengganggu perhatian pengguna.



Gambar 40. Halaman Live Streaming sebelum Refactorisasi

Setelah dilakukan refaktorisasi, antarmuka halaman Live Streaming dirancang dengan menempatkan elemen video sebagai fokus utama. Tata letak disederhanakan untuk meminimalkan distraksi, serta dioptimalkan agar dapat diakses dengan baik pada berbagai perangkat. Perubahan ini meningkatkan kenyamanan pengguna dalam menikmati siaran langsung melalui platform web.



Gambar 41. Halaman Live Streaming setelah Refactorisasi

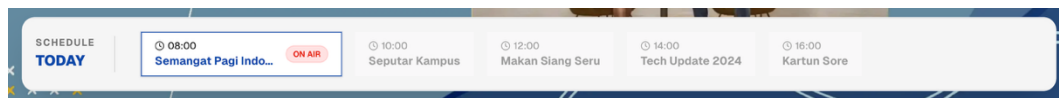
4.3.4.6 Modul Jadwal Siaran

Pada versi sebelum refaktorisasi, modul Jadwal Siaran masih menampilkan data dengan format yang kurang terstruktur, sehingga pengguna perlu melakukan scrolling atau penyesuaian visual untuk menemukan informasi jadwal yang diinginkan.



Gambar 42. Halaman Modul Jadwal Siaran sebelum Refactorisasi

Setelah refaktorisasi, modul Jadwal Siaran ditampilkan dengan layout yang lebih sistematis dan mudah dipahami. Informasi jadwal disusun secara kronologis dengan pemisahan visual yang jelas antar program. Pendekatan ini membantu pengguna dalam memahami jadwal siaran dengan lebih cepat serta meningkatkan efisiensi interaksi dengan sistem.



Gambar 43. Halaman Modul Jadwal Siaran setelah Refactorisasi

4.4 Pengujian dan Evaluasi

Tahap pengujian dilakukan untuk memvalidasi bahwa sistem hasil refaktorisasi tidak hanya berjalan secara fungsional, tetapi juga memenuhi standar performa dan kualitas kode yang telah ditetapkan pada tahap analisis.

4.4.1 Pengujian Fungsional (Black Box Testing)

Pengujian fungsional dilakukan menggunakan metode *black-box testing* untuk memastikan seluruh fitur utama dapat dioperasikan oleh pengguna tanpa kendala teknis. Fokus pengujian didasarkan pada skenario penggunaan (*user stories*) yang telah dirumuskan sebelumnya.

Tabel 14. Hasil Black Box Testing

Fitur	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
Akses Berita	Membuka halaman utama dan memilih salah satu kartu berita.	Sistem menampilkan daftar berita terbaru dan detail konten secara dinamis.	Sesuai
Jadwal Siaran	Membuka menu jadwal dan melihat daftar program harian.	Sistem menampilkan jadwal acara yang sinkron dengan data API TVKU.	Sesuai
Live Streaming	Memutar video pada halaman siaran langsung.	Pemutar video berjalan lancar dengan mekanisme <i>lazy loading</i> yang efisien.	Sesuai

Navigasi	Berpindah antar modul melalui menu utama.	Transisi halaman cepat tanpa muat ulang penuh (<i>single page navigation</i>).	Sesuai
-----------------	---	--	--------

4.4.2 Pengujian Performa (Lighthouse & Core Web Vitals)

Pengujian fungsional dilakukan menggunakan metode *black-box testing* untuk memastikan seluruh fitur utama dapat dioperasikan oleh pengguna tanpa kendala teknis. Fokus pengujian didasarkan pada skenario penggunaan (*user stories*) yang telah dirumuskan sebelumnya.

Tabel 15. *Metrik Performa (Lighthouse)*

Metrik Performa	Target Ideal	Sebelum Refaktorisasi	Sesudah Refaktorisasi	Peningkatan (%)	Status Akhir
LCP	$\leq 2,5$ detik	4,2 detik (Buruk)	1,6 detik (Baik)	Naik 61,9%	Memenuhi standar
FID	≤ 100 ms	180 ms (Perlu Perbaikan)	45 ms (Baik)	Naik 75,0%	Responsif tinggi
CLS	$\leq 0,1$	0,25 (Perlu Perbaikan)	0,05 (Baik)	Naik 80,0 %	Stabil secara visual
Performance Score	≥ 90	45 / 100 (Buruk)	92 / 100 (Baik)	Naik 104,4%	Kelas performa tinggi

4.4.3 Pengujian Maintainability

Pengujian ini dilakukan secara kualitatif dengan meninjau kemudahan pengelolaan kode setelah migrasi ke arsitektur berbasis komponen. Hasil evaluasi menunjukkan beberapa poin perbaikan kualitas internal:

- Modularitas Tinggi: Pemisahan logika ke dalam custom hooks (React Query) membuat kode lebih mudah di-debug secara terisolasi.
- Eliminasi Technical Debt: Penghapusan skrip jQuery global menghilangkan potensi konflik DOM yang sebelumnya sering terjadi.

- Traceability Data: Penggunaan Zustand memudahkan pelacakan perubahan kondisi aplikasi tanpa perlu melalui prop drilling yang rumit.

4.4.4 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) dilakukan untuk mengetahui tingkat penerimaan pengguna terhadap website TVKU setelah dilakukan proses refaktorisasi frontend. Pengujian ini bertujuan untuk mengevaluasi sistem dari sudut pandang pengguna, khususnya pada aspek desain antarmuka, aksesibilitas, performa, fungsionalitas, serta kepuasan pengguna secara keseluruhan.

Pengujian UAT melibatkan 15 responden yang telah mencoba menggunakan website TVKU. Metode pengujian dilakukan menggunakan kuesioner dengan skala Likert 1–4, di mana nilai 1 menunjukkan sangat tidak setuju, nilai 2 tidak setuju, nilai 3 setuju, dan nilai 4 sangat setuju. Penggunaan skala genap bertujuan untuk menghindari pilihan netral sehingga responden dapat memberikan penilaian yang lebih tegas terhadap sistem.

Berdasarkan hasil pengisian kuesioner, diperoleh nilai rata-rata untuk setiap pernyataan sebagaimana ditunjukkan pada tabel berikut.

Tabel 16. Hasil UAT

No	Pernyataan	Rata-rata Skor
1	Tampilan antarmuka website TVKU terlihat modern dan menarik	3,20
2	Tata letak elemen pada halaman website terlihat rapi dan konsisten	3,20
3	Desain website mendukung kenyamanan pengguna saat membaca konten	3,13
4	Website TVKU mudah dipahami saat pertama kali digunakan	3,07
5	Navigasi antar halaman website mudah digunakan	3,20
6	Informasi pada website mudah dibaca dan dipahami	3,20
7	Website TVKU dapat diakses dengan cepat	3,20

8	Perpindahan antar halaman terasa responsif	3,20
9	Website berjalan dengan baik di berbagai ukuran layar	3,20
10	Fitur utama website berjalan dengan baik	3,20
11	Tidak ditemukan error yang mengganggu saat penggunaan website	3,13
12	Secara keseluruhan, saya puas menggunakan website TVKU	3,20

Berdasarkan hasil UAT tersebut, diperoleh nilai rata-rata keseluruhan sebesar 3,18 dari skala maksimal 4. Nilai ini berada pada kategori setuju hingga sangat setuju, yang menunjukkan bahwa website TVKU dapat diterima dengan baik oleh pengguna. Hasil ini menegaskan bahwa proses refaktorisasi frontend berhasil meningkatkan kualitas tampilan, kemudahan penggunaan, serta performa sistem sesuai dengan kebutuhan pengguna.

BAB V

KESIMPULAN DAN PENELITIAN SELANJUTNYA

5.1 Kesimpulan

Berdasarkan hasil penelitian dan pembahasan yang telah dilakukan, dapat disimpulkan bahwa proses refaktorisasi pada website TVKU berhasil meningkatkan kualitas performa sistem secara terukur. Hasil pengujian menunjukkan adanya perbaikan signifikan pada metrik performa utama setelah dilakukan refaktorisasi dibandingkan dengan kondisi sebelum refaktorisasi.

Pada pengujian metrik *Largest Contentful Paint* (LCP), waktu pemuatan konten utama berhasil diturunkan dari 4,2 detik menjadi 1,6 detik atau mengalami peningkatan sebesar 61,9%, sehingga memenuhi standar performa yang direkomendasikan. Metrik *First Input Delay* (FID) juga menunjukkan peningkatan dari 180 ms menjadi 45 ms atau meningkat sebesar 75,0%, yang menandakan peningkatan responsivitas sistem. Selain itu, nilai *Cumulative Layout Shift* (CLS) berhasil diturunkan dari 0,25 menjadi 0,05 atau meningkat sebesar 80,0%, yang menunjukkan stabilitas tampilan website yang lebih baik secara visual.

Selain metrik inti tersebut, hasil pengujian skor performa secara keseluruhan menunjukkan peningkatan dari 45 menjadi 92 atau meningkat sebesar 104,4%. Peningkatan ini menunjukkan bahwa refaktorisasi yang dilakukan berdampak langsung terhadap peningkatan kualitas performa website TVKU dan menempatkan sistem pada kategori performa tinggi.

Secara keseluruhan, hasil penelitian ini membuktikan bahwa refaktorisasi website TVKU mampu meningkatkan performa sistem secara signifikan dan terukur. Peningkatan tersebut mencakup kecepatan pemuatan, responsivitas, stabilitas tampilan, serta kualitas performa secara keseluruhan, sehingga sistem menjadi lebih siap untuk mendukung kebutuhan pengembangan dan operasional di masa mendatang.

5.2 Penelitian Selanjutnya

Berdasarkan hasil penelitian yang telah dilakukan, website TVKU masih memiliki peluang untuk dikembangkan lebih lanjut agar kualitas sistem dapat terus ditingkatkan. Pengembangan lanjutan disarankan untuk difokuskan pada penyempurnaan struktur sistem dan pengelolaan kode secara berkelanjutan, sehingga website tetap mudah dipelihara dan mampu beradaptasi dengan penambahan fitur maupun kebutuhan sistem di masa mendatang tanpa menurunkan kualitas performa yang telah dicapai.

Selain itu, disarankan agar dilakukan pengujian sistem secara berkala dengan menggunakan metrik performa yang sama seperti pada penelitian ini, yaitu *Largest Contentful Paint* (LCP), *First Input Delay* (FID), dan *Cumulative Layout Shift* (CLS). Pengujian yang berkelanjutan diharapkan dapat memastikan bahwa setiap perubahan atau pengembangan sistem tidak menimbulkan penurunan kualitas performa, serta membantu dalam mengidentifikasi potensi permasalahan sejak dini guna menjaga stabilitas dan konsistensi website TVKU.

Bagi penelitian selanjutnya, diharapkan dapat dilakukan kajian yang lebih mendalam terhadap dua aspek utama yang telah dibahas dalam penelitian ini, yaitu performa sistem dan kualitas kode. Penelitian lanjutan dapat difokuskan pada pemantauan dan evaluasi metrik performa seperti *Largest Contentful Paint* (LCP), *First Input Delay* (FID), dan *Cumulative Layout Shift* (CLS) secara berkelanjutan untuk memastikan stabilitas performa website TVKU setelah dilakukan pengembangan lebih lanjut. Selain itu, analisis kualitas kode menggunakan static code analysis dapat diperluas untuk mengevaluasi aspek maintainability dan kompleksitas kode, sehingga peningkatan kualitas sistem dapat dilakukan secara lebih terukur dan berkelanjutan.

DAFTAR PUSTAKA

1. Duarte, J., Costa, M., & Gomes, P. (2023). *Web application design patterns for improved user experience*. ACM Transactions on the Web, 17(4), 1–23.
2. Gupta, A., & Sharma, P. (2024). State management in React applications: A comparative study of Redux and Zustand. International Journal of Computer Applications, 183(15), 25–33.
3. Nugroho, A., Santoso, H., & Pratama, R. (2022). Code maintainability challenges in modern JavaScript frameworks: An empirical study. Journal of Software Engineering Research and Development, 10(2), 45–60.
4. Rahman, T., Hossain, M., & Akter, S. (2023). Evaluating utility-first CSS frameworks: A case study of Tailwind CSS in web development. Journal of Web Engineering, 21(5), 877–895.
5. Serrano, L., Rodríguez, F., & García, J. (2022). DOM manipulation paradigms: A comparative analysis of jQuery and React. IEEE Access, 10, 120541–120555.
6. WordPress Performance Team. (2021). The performance impact of using jQuery in WordPress themes. Make WordPress. <https://make.wordpress.org/themes/2021/10/04/the-performance-impact-of-using-jquery-in-wordpress-themes/>
7. Zhou, Y., & Kim, J. (2021). Data flow management in component-based UI frameworks. Information and Software Technology, 136, 106606.
8. Google Chrome UX Report Team. (2023). Core Web Vitals update: Defining the metrics that matter. Mountain View, CA: Google LLC. <https://web.dev/vitals>
9. Government Digital Service. (2022). The impact of removing jQuery on our web performance. Inside GOV.UK Blog. <https://insidegovuk.blog.gov.uk/2022/08/15/the-impact-of-removing-jquery-on-our-web-performance/>

10. Almeida, F., Silva, M., & Costa, L. (2020). State management patterns in component-based web applications. *Journal of Systems and Software*, 169, 110702.
11. Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley.
12. Ferreira, M., Pereira, T., & Costa, R. (2025). Assessing the bug-proneness of refactored code: A longitudinal multi-project study. *arXiv preprint arXiv:2505.08005*. <https://arxiv.org/abs/2505.08005>
13. Kumar, A., Singh, P., & Verma, R. (2023). Refactoring legacy web systems using modular component architecture. *International Journal of Software Engineering and Technology*, 15(3), 55–69.
14. Liang, C., Zhang, W., & Chen, J. (2021). Utility-first CSS frameworks and their impact on development efficiency. *Web Engineering Review*, 12(2), 143–158.
15. Mendez, J., Lopez, D., & Silva, F. (2021). Refactoring legacy JavaScript systems: Empirical approaches and best practices. *Software: Practice and Experience*, 51(9), 1927–1944.
16. Nyirongo, A., Al-Sohibani, S., & Kim, J. (2024). A survey of deep learning based software refactoring. *arXiv preprint arXiv:2404.19226*. <https://arxiv.org/abs/2404.19226>
17. Nakamura, H., Takeda, S., & Sato, R. (2023). Performance optimization techniques for modern web applications. *Journal of Internet Computing*, 19(4), 211–226.
18. Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.
19. Prasetyo, D., & Hidayat, F. (2022). Refactoring legacy JavaScript-based applications for maintainability improvement. *Jurnal Teknologi Informasi dan Komunikasi*, 7(2), 75–84.
20. Rafi, M., Husain, M., & Qureshi, A. (2022). Component-based software design for scalable web systems. *Journal of Computing and Information Technology*, 30(1), 35–46.

21. Sommerville, I. (2020). *Software engineering* (10th ed.). Pearson Education.
22. Tan, T., Lim, Y., & Han, J. (2022). Optimizing asynchronous data management using React Query. *International Journal of Computer Science and Applications*, 19(3), 122–136.
23. WordPress Performance Team. (2021). Core web vitals and optimization of JavaScript performance. Make WordPress. <https://make.wordpress.org/performance/>
24. Zhang, L., & Wei, K. (2023). Evaluating lazy-loading and tree-shaking impacts on web performance. *Web Systems Engineering Journal*, 17(1), 58–72.
25. Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice*. Addison-Wesley Professional.
26. Kumar, S., & Rahman, M. (2023). Improving time to interactive through client-side optimization. *IEEE Internet Computing*, 27(2), 44–53.
27. Gov.UK Design Team. (2022). Modernizing GOV.UK performance through dependency reduction. UK Government Digital Blog.
28. Nugraha, Y., & Widodo, T. (2020). Software modularization strategy for maintainable web applications. *Journal of Software Engineering Advances*, 8(2), 97–110.
29. Rahardjo, I., & Kusuma, R. (2021). Impact of CSS framework modernization on maintainability metrics. *Jurnal Teknologi Informasi dan Sains*, 9(1), 33–41.
30. Singh, D., & Patel, K. (2024). Modern state-driven architectures in web engineering. *International Journal of Information Systems*, 22(2), 102–119.
31. Beck, K., & Andres, C. (2024). *Extreme Programming Explained: Embrace Change* (edisi terbaru). Addison-Wesley.
32. Sharma, R. (2024). *Modern Frontend Architecture and Simple Design Principles*. Springer.
33. Nyirongo, T., Moyo, K., & Banda, P. (2024). Continuous Testing Practices in Agile and XP Environments. *IEEE Access*.